Movecity
Documento de Visão

1. Introdução

O presente documento formaliza a visão estratégica do projeto **Movecity**, desenvolvido como Trabalho de Conclusão de Curso. O objetivo é apresentar a viabilidade técnica e social desta solução a possíveis stakeholders, sendo estes gestores públicos, empresas de transporte que operam no Distrito Federal ou investidores. O projeto é conduzido pelo grupo Segurança no Transporte, integrado pelos acadêmicos: Breno Santana Silva, Davi Eduardo Costa Miranda, Kelvin Rodrigues de Sousa, Luis Fernando Monteiro de Assis Lourenço, Nathalia Gualberto Lopes e Vitória Cordeiro Albuquerque.

O Movecity é um novo software, desenvolvido como um aplicativo web. O cliente é acadêmico: não há cliente externo contratante nem fins lucrativos. Diferente de soluções globais, o Movecity nasce com foco regionalizado, operando como projeto-piloto em duas cidades satélites do Distrito Federal. A escolha de regiões de alta densidade demográfica, como Ceilândia e Taguatinga — identificadas pela Pesquisa Distrital por Amostra de Domicílios (IPEDF, 2022) como áreas de grande concentração populacional e intensa demanda por transporte público —, é fundamental para a validação da solução em um cenário de alta complexidade. A motivação do projeto é mitigar as falhas críticas de comunicação do sistema de transporte local, promovendo segurança e previsibilidade para o passageiro brasiliense por meio de uma plataforma colaborativa e transparente.

2. Contexto de negócio

2.1 Cenário Atual do Domínio e Deficiências

O domínio de aplicação deste projeto concentra-se na gestão e consumo de informações sobre a mobilidade urbana no Distrito Federal. O sistema de transporte público do DF é caracterizado por um fluxo pendular intenso, onde milhares de cidadãos se deslocam diariamente das Regiões Administrativas (como Ceilândia e Taguatinga) para o Plano Piloto e outros centros econômicos.

Atualmente, este cenário é marcado por uma profunda assimetria de informação. Embora existam frotas monitoradas, o dado real sobre a localização dos veículos raramente é transmitido de forma fidedigna ao cidadão. As principais lacunas enfrentadas são:

- **O fenômeno do "Ônibus Fantasma":** veículos que constam no sistema oficial mas não realizam o percurso, ou que passam fora do horário previsto sem aviso prévio.
- **Insegurança física e psicológica:** a espera prolongada em paradas de ônibus, muitas vezes em locais isolados e com iluminação precária nas cidades satélites, expõe o usuário a riscos de assaltos e violência.
- **Ineficiência operacional e econômica:** a falta de previsibilidade impede que o usuário otimize seu tempo, impactando sua pontualidade no trabalho e compromissos.

2.2 Usuários, Mercado-Alvo e Indicadores de Relevância

O mercado-alvo compreende os usuários dependentes do STPC/DF. O projeto-piloto é em Taguatinga e Ceilândia devido à alta densidade demográfica. Segundo o IPEDF (2022), 33,3% da população do DF utiliza o ônibus como principal meio de transporte.

Principais usuários:

- Trabalhadores de turnos críticos.
- Estudantes universitários e de ensino técnico.
- Profissionais corporativos e autônomos.

2.3 Benchmarking

Ferramenta	Pontos Fortes	Limitações	Oportunidade Movecity
DF no Ponto	Dados diretos do GDF.	Baixa usabilidade e bugs.	UX superior e validação ágil.
Moovit/Maps	Grande alcance.	Dependência de feeds lentos.	Precisão regional cirúrgica.
Redes Sociais	Colaboração real.	Informação desorganizada.	Colaboração georreferenciada.

3. Posicionamento

3.1 Declaração do problema

O problema de	A ausência de informações confiáveis e em tempo real sobre o transporte público no DF
afeta	Trabalhadores e estudantes de Ceilândia e Taguatinga
cujo impacto é	Insegurança física, perda de produtividade e ansiedade
uma solução de sucesso deveria	Fornecer rastreamento real, alertas de atrasos e permitir reportes colaborativos

3.2 Declaração da visão do software

Para	Usuários do transporte público do DF
Que	Necessitam de informações precisas para planejar deslocamentos com segurança
O	Movecity
É um	Aplicativo web de mobilidade urbana colaborativa
Que	Combina dados de GPS com informações colaborativas dos passageiros
Diferente de	DF no Ponto, Moovit e Google Maps
Nosso produto	Entrega maior precisão e foco regional nas cidades satélites

4. Descrição das partes interessadas

Nome	Descrição	Responsabilidades
Usuário Trabalhador	Depende do transporte em horários críticos	Reportar ocorrências e validar dados
Equipe de Desenvolvimento	Grupo Segurança no Transporte (UCB)	Desenvolver e manter a plataforma
Gestores Públicos	SEMOB/GDF	Consumir dados para tomada de decisão

5. Visão geral do produto

5.1 Necessidades e funcionalidades

O detalhamento de cada requisito, com critérios de aceite, está no DRS. Responsáveis conforme distribuicao_us.md.

Necessidade	Funcionalidade	Prioridade	Responsável
Saber onde está o ônibus (combate ao "ônibus fantasma")	Mapa com a localização do usuário, busca de linha por número, trajeto e paradas,	Alta	@Kelvin963 (#14–#16), @davieduardo001 (#17), @ngualbertolopes (#199)

Necessidade	**Funcionalidade**	**Prioridade**	**Responsável**
	posição dos ônibus atualizada sozinha (US #14, #15, #16, #17) e, quando o ônibus não aparece no GPS oficial, posição informada por passageiros a bordo, com consentimento (US #199)		
Saber quanto falta para o ônibus chegar	Tempo estimado de chegada (ETA) no ônibus, na parada e na rota, sempre com a origem do número: ao vivo, previsto ou "sem previsão" (US #19, #159, #173)	Alta	@louisassis (#19), @Vitoria-Albuquerque (#159), @davieduardo001 (#173)
Chegar de um ponto a outro	Rota por ônibus com baldeação, paradas de subir e descer, busca de parada por nome ou endereço e detalhes da parada (US #18, #20, #21, #190)	Alta	@louisassis (#18, #20, #21), @davieduardo001 (#190)
Saber a que horas sair de casa	Planejamento por horário: "sair às 7h10" ou "chegar até 8h00", com o horário de cada ônibus e a espera na baldeação (US #115)	Média	@davieduardo001
Acompanhar a viagem e não perder a parada	"Peguei este ônibus", acompanhamento em viagem e alerta de proximidade, dentro do app e por Web Push com a tela bloqueada (US #171, #172, #195)	Alta	@Kelvin963 (#171, entregue por @Vitoria-Albuquerque), @gualbertonathalia (#172, concluída por @davieduardo001), @davieduardo001 (#195)
Ser avisado de atraso e das rotas de interesse	Alerta de atraso, notificações de rotas preferidas, alerta de cancelamento de viagem e preferências de notificação (US #22, #27, #28, #29)	Alta	@gualbertonathalia (#22, #27, #29), @Kelvin963 (#28)
Contribuir com informação de campo	Reporte de ocorrência, lista de ocorrências e confirmação por outros passageiros, estilo Waze (US #23, #24, #26)	Média	@Vitoria-Albuquerque
Voltar rápido a trajetos frequentes	Rota favorita (US #25)	Média	@Vitoria-Albuquerque
Acesso seguro e simples	Cadastro, login por e-mail e senha ou Google, recuperação de senha por e-mail e sessão com renovação automática (US #10, #11, #12, #13, #31, #133)	Alta	@brenouchihar (#10, #11, #13, #133), @davieduardo001 (#12, #31)
Aprender a usar e ter à mão	Tutorial no primeiro acesso e instalação como aplicativo (PWA) (US #30, #136)	Média	@Vitoria-Albuquerque (#30), @davieduardo001 (#136)

5.2 Arquitetura Técnica

Camada	Tecnologia	Justificativa
Frontend	Next.js (React), na Vercel	SSR/SSG, App Router, deploy automático a cada push
Mapa/GPS	Leaflet JS + OpenStreetMap (base CARTO Voyager)	Gratuito, open source e com alta customização
Backend	FastAPI (Python) no Fly.io, em quatro apps: gateway, auth, mobilidade e colaboração	Framework assíncrono, documentação automática via OpenAPI, deploy via Docker
Banco de Dados	PostgreSQL (Neon), sem PostGIS	Serverless e free tier generoso. O PostGIS foi avaliado e não adotado: as geometrias ficam em colunas JSON (ver Documento de Arquitetura, ADR-02)
CI/CD	GitHub Actions	Integração nativa com o repositório; testes e deploy automatizados
Fonte GPS	API do GDF (SEMOB)	Dados oficiais das frotas do STPC/DF em GeoJSON: catálogo de linhas copiado por ingestão em lote, posição lida ao vivo e nunca guardada

Fluxo de dados:

1. A API do GDF (SEMOB) fornece as posições GPS dos ônibus; o backend as lê ao vivo (cache de 20 s) e não as grava.
2. O backend combina a posição ao vivo com o catálogo de linhas e paradas para calcular ETA, rotas e viagem. Quando uma linha não tem ônibus no feed, a posição informada por passageiros a bordo, com consentimento, entra no lugar, sempre com o selo "colaborativo"; a posição da SEMOB sempre vale mais. O planejamento por horário aplica a tabela de saídas da SEMOB às opções de rota. Os reportes dos usuários ficam em tabela própria e aparecem na lista de ocorrências; não alteram o ETA nem a posição dos ônibus.
3. O frontend renderiza as informações no mapa via Leaflet JS.
4. Os reportes colaborativos são validados por cruzamento: atraso, não passou, lotação, obra na via e ônibus quebrou ficam "aguardando confirmação" até receberem **3 confirmações de outros passageiros**; acidente e segurança aparecem confirmados de imediato. O autor não confirma o próprio reporte.

5.3 Requisitos não funcionais preliminares

Requisito	Descrição	Prioridade	Situação
Desempenho	Carregamento em até 3 s em conexões 4G	Alta	Meta; não medida em rede 4G.
Disponibilidade	99% nos horários de pico (5h–9h, 17h–20h)	Alta	Meta; sem monitoramento de uptime.
LGPD	Proteção rigorosa de dados de geolocalização	Alta	Nenhuma coordenada do usuário vai para o banco: a viagem não tem coluna de posição, a posição enviada no embarque é descartada e nenhuma coordenada vai para o endereço das páginas. A única exceção é a posição colaborativa (US #199), só

Requisito	Descrição	Prioridade	Situação
			com consentimento explícito e revogável: ela fica na memória do servidor por até 2 minutos e nunca é gravada; no banco ficam só o instante e a versão do texto aceito. Sem auditoria formal.
Validação cruzada	Reporte colaborativo precisa de confirmação de outros passageiros	Média	Implementada com 3 confirmações (o planejamento inicial previa 2).

5.4 Restrições e premissas

- **Sem PostGIS e sem histórico de posições:** busca por raio no servidor ou análise retrospectiva exigiriam reavaliar essas decisões.
- **Dados da SEMOB** por ingestão mensal em lote e posição ao vivo; os endpoints são públicos, sem documentação nem SLA.
- **Sem ambiente de homologação separado**, por custo: a homolog publica na produção da Vercel, e backend e banco são compartilhados.
- **Plataforma:** aplicativo web instalável (PWA); no iPhone, o aviso com a tela bloqueada exige instalar o app na tela inicial.
- **Posição colaborativa:** as leituras ficam na memória do processo do serviço de mobilidade. Com mais de uma máquina no Fly.io, o fallback pode não aparecer para quem consulta em outra máquina; trocar por armazenamento compartilhado é uma mudança só de implementação.
- **Horário planejado:** o horário de passagem em cada parada é estimado (saída do terminal pela tabela da SEMOB mais o tempo de percurso); feriados não são considerados porque a SEMOB não informa.
- **Ajuda ao usuário:** o tutorial do primeiro acesso é o único material de uso dentro do app.

6. Escopo entregue

Estão entregues todas as funcionalidades da tabela 5.1: acesso, mapa e rastreamento, ETA, rotas, planejamento por horário, parada, viagem, ocorrências, rota favorita, tutorial, alertas (atraso, proximidade e cancelamento) e instalação como PWA. Além do planejado no início, o produto ganhou "pegar o ônibus" (#171), alerta de proximidade (#172) por Web Push (#195), paradas de subir e descer na rota (#190), ETA pela parada física (#173), PWA (#136), e-mail de recuperação de senha (#237), alerta de cancelamento de viagem (#28), planejamento por horário (#115) e posição colaborativa com consentimento (#199).

Não foi feita validação em campo: o raio de embarque, a tolerância do ETA e a regra "ônibus chegando" não foram medidos com um ônibus real. O texto dos Termos de Uso e da Política de Privacidade (#32) ainda precisa ser publicado numa tela do aplicativo; hoje o cadastro exige apenas o aceite. Os resultados dos testes estão no Documento de Testes e Implementação.

7. Referências

- IPEDF (2022). Pesquisa Distrital por Amostra de Domicílios.
- CGDF (2026). Painel de Ouvidoria do Distrito Federal.
- CAROLI, P. (2018). Lean Inception.

8. Histórico de revisões

Versão	Data	Alteração
1.0	30/10/2026	Versão entregue, refletindo o produto implementado

Versão	**Data**	**Alteração**
1.1	31/10/2026	Inclusão do alerta de cancelamento (#28), do planejamento por horário (#115) e da posição colaborativa (#199)

Movecity
Documento de Requisitos de Software

Versão 1.1 · 31/10/2026. *Descreve o produto* ***como está entregue****, não como foi planejado. "Entregue" significa mergeado em homolog ou main; o código em produção é o de homolog. Estrutura conforme o Template_DRS da disciplina.*

1. Introdução

1.1 Propósito do documento

Definir e consolidar os requisitos do Movecity (aplicativo web de mobilidade urbana colaborativa para o DF, projeto-piloto em Taguatinga e Ceilândia) para servir de base ao projeto, aos testes e à manutenção. Cada requisito aponta para a User Story (issue [US]) de origem.

1.2 Link para o Documento de Visão

docs/documento_visao.md.

1.3 Definições, acrônimos e abreviações

Termo	Significado
US	User Story, registrada como issue [US] no GitHub, com cenários BDD (Dado/Quando/Então)
BDD	Behavior-Driven Development; formato dos critérios de aceite
SEMOB	Secretaria de Mobilidade do DF; fonte dos dados de linhas, paradas e posição GPS dos ônibus
ETA	Tempo estimado de chegada (*estimated time of arrival*)
Gateway	Ponto único de entrada das requisições do frontend; valida o JWT e roteia para Auth, Mobilidade e Colaboração
PWA	Aplicativo web instalável na tela inicial
Web Push	Notificação entregue ao aparelho com o app fechado ou a tela bloqueada
LGPD	Lei Geral de Proteção de Dados
RF / RNF / RN	Requisito funcional / requisito não funcional / regra de negócio
ADR	Registro de decisão arquitetural, no Documento de Arquitetura, seção 10

1.4 Metodologia de elicitação

- **Lean Inception** (CAROLI, 2018), que originou o Documento de Visão e o backlog.
- **Histórias de usuário com cenários BDD**, escritas como issues [US] e refinadas ao longo das sprints.
- **Protótipo de alta fidelidade** (issue #33), adaptado depois à identidade visual v3.
- **Pesquisa de integração** com os dados de ônibus do DF.
- **Teste real e execução de casos manuais em produção**, que geraram defeitos e requisitos corrigidos.

2. Visão geral dos requisitos funcionais

Abordagem: histórias de usuário (US), com a lista de casos de uso (diagramas de sequência) na seção 6. O backlog é o conjunto de issues [US] do GitHub. A prioridade (Alta, Média ou Baixa) é a do Documento de Visão, mantida

por funcionalidade, sem técnica formal de priorização (MoSCoW, WSJF).

Necessidade (Visão)	**Requisitos**
Saber onde está o ônibus	RF-06 a RF-09, RF-30
Saber quanto falta	RF-10, RF-11
Chegar de um ponto a outro	RF-12, RF-13, RF-14, RF-29
Acompanhar a viagem	RF-15, RF-16, RF-17
Contribuir com informação de campo	RF-18 a RF-20
Ser avisado de atraso e rotas	RF-22, RF-23, RF-24, RF-28
Acesso seguro	RF-01 a RF-05
Voltar a trajetos frequentes; aprender a usar	RF-21, RF-25, RF-26, RF-27

São **30 requisitos funcionais**, todos entregues.

3. Especificação dos requisitos funcionais

Responsáveis conforme distribuicao_us.md. Para as USs criadas depois do plano, o responsável é o autor da issue e o atribuído.

3.1 Acesso (autenticação e sessão)

RF-01 — Criar nova conta · US #11 · Alta · @brenouchihar

- Como novo passageiro, quero criar uma conta com nome, e-mail e senha, para personalizar minha experiência.
- Aceite: (1) cadastro com todos os campos cria a conta; (2) campo obrigatório ausente é recusado; (3) e-mail já cadastrado é recusado; (4) a senha precisa ter 8 caracteres, maiúscula, número e símbolo, e a tela mostra o medidor; (5) sem aceite dos termos, o cadastro é recusado.
- Notas: não há confirmação de conta por e-mail; a conta entra direto.

RF-02 — Login com e-mail e senha · US #10 · Alta · @brenouchihar

- Aceite: (1) credenciais corretas autenticam e levam ao mapa; (2) senha incorreta mostra "Usuário ou senha inválidos"; (3) a sessão persiste por cookies httpOnly.
- Notas: a conta é bloqueada por 15 min após 5 tentativas falhas.

RF-03 — Login com Google · US #12 · Média · @davieduardo001

- Aceite: no primeiro acesso via Google a conta é criada e o usuário entra.

RF-04 — Recuperação de senha · US #13 e #133 · Média · @brenouchihar

- Aceite: (1) solicitar a redefinição por e-mail; (2) a resposta é a mesma para e-mail existente e inexistente, sem revelar quem tem conta; (3) excesso de tentativas é limitado; (4) token válido redefine a senha; (5) token inválido ou expirado é recusado; (6) confirmação de senha divergente é recusada.
- Notas: o link vale 60 minutos, uso único; o e-mail é enviado por SMTP.

RF-05 — Ciclo de vida da sessão · US #31 · Alta · @davieduardo001

- Aceite: (1) recurso protegido com sessão válida responde normalmente; (2) sessão expirada é renovada automaticamente com o refresh token; (3) sessão inválida sem renovação volta ao login; (4) logout encerra a sessão.
- Notas: access token de 1 h e refresh token de 7 dias, em cookies `HttpOnly`; `Secure`; `SameSite=None`. O Gateway ignora o `X-User-Id` enviado pelo cliente.

3.2 Mapa, linha e rastreamento

RF-06 — Mapa com localização atual · US #14 · Alta · @Kelvin963

- Aceite: (1) com permissão, o mapa centra na posição do usuário; (2) com a localização negada, aparece "Não foi possível obter sua localização"; (3) o marcador acompanha a movimentação.

RF-07 — Buscar linha por número · US #15 · Alta · @Kelvin963

- Aceite: (1) a busca devolve a linha com nome, sentido e itinerário; (2) linha inexistente mostra erro; (3) a resposta vem em tempo adequado. A busca aceita também nome ou destino (autocomplete).

RF-08 — Rastrear a posição do ônibus em tempo real · US #16 · Alta · @Kelvin963

- Aceite: (1) ícones dos ônibus sobre o trajeto; (2) atualização automática sem recarregar; (3) sem veículo em operação, aviso e próxima saída prevista do terminal.
- Notas: posição lida ao vivo da SEMOB com cache de 20 s, nunca guardada.

RF-09 — Trajeto e paradas da linha · US #17 · Alta · @davieduardo001

- Aceite: (1) trajeto desenhado pela malha viária; (2) paradas marcadas, com nome ao tocar; (3) seta indica o sentido de operação.

RF-10 — Detalhes de uma parada · US #18 e #174 · Média · @louisassis

- Aceite: (1) painel com nome, código, linhas e próximas saídas; (2) parada sem horários mostra mensagem; (3) fechar o painel volta ao mapa.
- Notas: a seção "Chegando nesta parada" fica oculta durante uma viagem, em erro ou com lista vazia.

RF-11 — Tempo estimado de chegada (ETA) · US #19, #159 e #173 · Alta · @louisassis (#19), @Vitoria-Albuquerque (#159), @davieduardo001 (#173)

- Aceite: (1) ETA calculado pela posição do veículo; (2) atualizado sem recarregar; (3) sem dado, mostra "Sem previsão no momento"; (4) ETA exibido no marcador do ônibus; (5) marcador sem ETA calculável fica sem número.
- Notas: todo tempo exibido declara a origem (ao vivo ou previsto); o ETA é medido até a parada física, no sentido dela.

RF-12 — Calcular rota de origem a destino por ônibus · US #20 e #159 · Alta · @louisassis

- Aceite: (1) rota simples; (2) rota com baldeação e ponto de troca; (3) nenhuma rota: mensagem; (4) destino por endereço ou ponto de referência (geocodificação).

RF-13 — Buscar parada por nome ou endereço · US #21 · Média · @louisassis

- Aceite: (1) busca por nome com resultado; (2) busca por endereço mostra as paradas num raio de 500 m; (3) sem resultado, mensagem orientando a tentar outro nome, bairro ou ponto de referência.

RF-14 — Paradas de subir e descer na rota, com o tempo do ônibus · US #190 · Média · @davieduardo001

- Aceite: (1) só as paradas de embarque e desembarque de cada trecho em destaque; (2) tempo do ônibus na parada de embarque, com a origem; (3) tempo até a parada de desembarque; (4) sem ônibus, horário previsto ou "Sem previsão no momento"; (5) baldeação mostra as paradas de cada trecho; (6) "Peguei este ônibus" no trecho já define o destino; (7) ponto sem parada cadastrada aparece como "Parada sem nome"; (8) "chega na sua parada em N min" no popup do ônibus.

RF-29 — Planejar a viagem por horário de partida ou chegada · US #115 · Média · @davieduardo001

- Aceite: (1) "sair às X" mostra as opções com o horário de cada trecho; (2) "chegar até X" calcula o horário de saída necessário; (3) a baldeação mostra o tempo de espera no ponto de troca; (4) usa a tabela do dia da semana escolhido; (5) sem viagem possível, explica o motivo (sem linha para o trajeto, sem tabela de horários ou fora do horário de operação), em vez de uma lista vazia.
- Notas: parte das opções do RF-12; o horário de passagem em cada parada é **estimado** (saída do terminal pela tabela da SEMOB mais o tempo de percurso, proporcional ao ponto do trajeto) e a tela avisa isso; feriados não são considerados; espera acima de 3 h não conta como viagem.

RF-30 — Posição colaborativa do ônibus (fallback do GPS oficial) · US #199 · Alta · @ngualbertolopes

- Aceite: (1) ao pegar o ônibus, a pessoa pode aceitar compartilhar a posição do ônibus; o compartilhamento vem **desligado**, separado do aceite dos Termos; (2) com o ônibus fora do GPS da SEMOB, a posição

informada entra no mapa e no ETA com o selo "colaborativo"; (3) "Parar" revoga a qualquer momento e apaga a leitura na hora; "Cheguei" também encerra o compartilhamento; (4) leituras fora do traçado da linha, com salto de velocidade impossível ou enviadas com menos de 10 s de intervalo são recusadas; (5) quando o ônibus volta ao feed da SEMOB, a posição oficial vale.

- Notas: nenhuma coordenada vai para o banco; cada leitura vive só na memória do servidor, por no máximo 2 minutos (RN-024). O banco guarda apenas o instante e a versão do texto aceito.

3.3 Viagem e alertas de proximidade

RF-15 — Pegar o ônibus (embarcar, acompanhar e encerrar a viagem) · US #171 · Alta · @Kelvin963 (atribuída), entregue por @Vitoria-Albuquerque

- Aceite: (1) "Peguei este ônibus" registra a viagem e ativa o modo "em viagem"; (2) em viagem, mostra o veículo, a próxima parada e as paradas restantes; (3) "Cheguei" encerra; (4) sem veículo identificável, avisa e não registra.
- Notas: uma viagem ativa por usuário; a viagem expira em 4 h; o embarque exige um ônibus da linha por perto, e a posição do usuário só valida o embarque e é descartada.

RF-16 — Alerta de proximidade da parada (dentro do app) · US #172 · Alta · @gualbertonathalia, concluída por @davieduardo001

- Aceite: (1) alerta quando o ônibus chega à distância ou tempo configurado; (2) uma única vez por parada e viagem; (3) respeita as preferências (RF-24); (4) sem posição do veículo, não emite alerta impreciso.

RF-17 — Alerta de proximidade com a tela bloqueada (Web Push) · US #195 · Alta · @davieduardo001

- Aceite: (1) ativar com consentimento explícito em Configurações; (2) receber com o app fechado, e o toque abre o mapa; (3) não receber duas vezes; (4) respeita as preferências; (5) remover a inscrição; (6) servidor sem chaves de push: controle indisponível, sem erro; (7) no iPhone, exige instalar o app (PWA); (8) permissão negada é explicada.

3.4 Colaboração

RF-18 — Reportar ocorrência em uma linha · US #23 · Média · @Vitoria-Albuquerque

- Aceite: (1) reporte com sucesso é registrado e exibido; (2) sem autenticação, pede login; (3) limite de **3 reportes por hora** por usuário, com aviso.
- Notas: tipos atraso, não passou, lotação, obra na via, ônibus quebrou, acidente e segurança.

RF-19 — Ver ocorrências reportadas por outros passageiros · US #24 · Média · @Vitoria-Albuquerque

- Aceite: (1) ocorrência confirmada exibida; (2) pendente exibida como "Aguardando confirmação"; (3) expirada some da lista.
- Notas: a lista não mostra quem reportou; a validade varia por tipo (RN-014).

RF-20 — Confirmar ocorrência de outro passageiro · US #26 · Média · @Vitoria-Albuquerque

- Aceite: (1) confirmação de ocorrência pendente; (2) o autor não confirma a própria; (3) não é possível confirmar após a expiração; (4) cada usuário confirma uma vez; (5) com **3 confirmações** a ocorrência vira "confirmada".

RF-21 — Salvar e visualizar rota favorita · US #25 · Média · @Vitoria-Albuquerque

- Aceite: (1) salvar a rota; (2) acesso rápido: "Ir" recalcula com origem e destino preenchidos; (3) remover. Máximo de 20 favoritos por usuário.

3.5 Notificações

RF-22 — Notificações de rotas preferidas · US #22 · Alta · @gualbertonathalia

- Aceite: (1) notificação de ônibus próximo; (2) com a antecedência configurada; (3) desabilitada respeita a preferência.

RF-23 — Alerta de atraso na linha acompanhada · US #27 · Alta · @gualbertonathalia

- Aceite: (1) alerta automático acima do limiar de atraso; (2) atualizado quando o atraso aumenta; (3) cancelado quando o ônibus normaliza.

RF-24 — Preferências de notificação · US #29 · Média · @gualbertonathalia

- Aceite: (1) antecedência de 5, 10, 30 ou 60 min; (2) desativar um tipo; (3) desativar todas.

RF-28 — Alerta de cancelamento de viagem · US #28 · Alta · @Kelvin963

- Aceite: (1) a linha acompanhada (favoritos e linhas seguidas) sem ônibus no feed 15 minutos depois do horário previsto de saída gera um alerta de cancelamento; (2) o alerta sugere uma linha alternativa quando outra cobre ao menos 70% das paradas do trecho; (3) respeita a preferência de notificação de cancelamento (RF-24); (4) com o feed inteiro da SEMOB vazio, não alerta.
- Notas: detecta o cancelamento pela **ausência** do ônibus; a SEMOB não informa cancelamento operacional, então esse caso não existe. A notificação sai por Web Push (RF-17).

3.6 Onboarding e instalação

RF-25 — Tutorial no primeiro acesso · US #30 · Média · @Vitoria-Albuquerque

- Aceite: (1) tutorial automático no primeiro acesso; (2) "Pular" não o reabre; (3) rever pelas Configurações.

RF-26 — Instalar o Movecity na tela inicial (PWA) · US #136 · Média · @davieduardo001

- Aceite: (1) instalar pelo Android; (2) instalar pelo iPhone; (3) abre no mapa; (4) dado de transporte nunca vem de cache; (5) oscilação de rede não quebra a casca do app; (6) ícone recortado pelo sistema.

RF-27 — Navegação sem itens mortos · issue #201 (UX) · Baixa · @davieduardo001

- Aceite: a barra lateral não mostra "Nova Viagem".

4. Lista de regras de negócio

ID	Descrição	Requisitos
RN-001	O passageiro está **Aguardando** ou **Em viagem**; só assume "Em viagem" por indicação explícita confirmada no servidor, nunca por suposição	RF-15
RN-002	Em viagem, o ETA de espera some; mostram-se a próxima parada e as paradas restantes	RF-15
RN-003	LGPD na viagem: guarda-se o mínimo (usuário, linha, sentido, veículo, parada de destino, horários, e o aceite do compartilhamento quando houver), **sem trilha de posição**; a posição do usuário valida o embarque e é descartada; nenhuma coordenada do usuário vai para o endereço da página	RF-12, RF-15, RF-30
RN-004	Confirmar o embarque exige veículo **da linha**, com posição recente (até 5 min), a até o raio de embarque do usuário (padrão 100 m); com vários, vale o mais próximo	RF-14, RF-15
RN-005	Uma viagem ativa por usuário, garantida por índice único no banco	RF-15
RN-006	Viagem esquecida expira sozinha em 4 h e não gera alerta	RF-15, RF-16
RN-007	O ETA é medido ao longo do trajeto até	RF-10, RF-11

ID	Descrição	Requisitos
	a **parada física**, no sentido dela; paradas de lados opostos são diferentes	
RN-008	Horário previsto nunca mistura dia útil, sábado e domingo	RF-10, RF-11
RN-009	**Não inventar:** sem posição válida (até 15 min) nem horário confiável, a resposta é "sem previsão"	RF-11, RF-14, RF-16
RN-010	Todo número de tempo declara a origem: ao vivo, previsto ou saída do terminal; saída do terminal nunca é exibida como "passa aqui"	RF-10, RF-11, RF-14
RN-011	Ônibus que já passou, de sentido oposto ou com dado velho não entra como "chegando"	RF-11
RN-012	Posição do ônibus lida ao vivo (cache de 20 s) e nunca guardada	RF-08, RF-11
RN-013	Limite de **3 reportes por hora** por usuário	RF-18
RN-014	Validade da ocorrência por tipo: lotação 15 min; atraso, acidente e segurança 30 min; não passou e ônibus quebrou 60 min; obra na via 120 min. Expirada não aparece; o registro não é apagado	RF-19
RN-015	Ocorrência dos tipos que exigem validação nasce "pendente" e vira "confirmada" com **3 confirmações** de outros usuários; acidente e segurança nascem confirmadas	RF-18, RF-19, RF-20
RN-016	O autor não confirma a própria ocorrência; um usuário confirma uma vez; ocorrência expirada não é confirmada	RF-20
RN-017	Máximo de 20 rotas favoritas por usuário	RF-21
RN-018	Antecedência do alerta: 5, 10, 30 ou 60 min (padrão 30); notificações e alerta de chegada são ativáveis separadamente	RF-16, RF-17, RF-24
RN-019	O alerta de proximidade dispara uma vez por parada e viagem, respeita as preferências e não dispara sem posição do veículo	RF-16, RF-17
RN-020	Conta bloqueada por 15 min após 5 tentativas de login falhas	RF-02
RN-021	Cadastro exige aceite dos termos; a recuperação de senha responde igual para e-mail existente e inexistente	RF-01, RF-04
RN-022	O servidor ignora o X-User-Id enviado pelo cliente; a identidade vem do JWT validado no Gateway	todas as rotas protegidas

ID	Descrição	Requisitos
RN-023	O compartilhamento da posição do ônibus é **opt-in**, desligado por padrão e separado do aceite dos Termos; exige a versão do texto aceito, que fica guardada como prova; é revogável a qualquer momento ("Parar") e termina com "Cheguei"	RF-30
RN-024	A posição colaborativa nunca é gravada: cada leitura vale até 2 minutos, só em memória, sem identificador do usuário; o banco guarda apenas o instante do aceite e a versão do texto	RF-30
RN-025	A posição da SEMOB sempre vence a colaborativa (no empate de ETA também); a colaborativa só entra quando não há ônibus da SEMOB equivalente, e o selo declara a origem	RF-30, RF-11
RN-026	Leitura colaborativa só é aceita se encaixar no traçado da linha (150 m), sem salto acima da velocidade máxima de ônibus e a cada 10 s no mínimo por viagem	RF-30
RN-027	No planejamento por horário, cada trecho usa a tabela do dia da semana (nunca mistura dia útil, sábado e domingo); sem opção viável, a resposta diz o motivo	RF-29, RN-008
RN-028	O cancelamento só é declarado quando a linha está sem ônibus e outras linhas reportam posição no mesmo ciclo; com o feed inteiro vazio, a causa pode ser a fonte, e não se alerta	RF-28, RN-009

5. Lista de requisitos não funcionais

Classificação ISO/IEC 25010.

ID	Requisito	Classificação	Situação
RNF-01	Carregamento em até 3 s em conexão 4G	Eficiência de desempenho	Meta; não medida
RNF-02	Disponibilidade de 99% nos horários de pico (5h–9h, 17h–20h)	Confiabilidade	Meta; sem monitoramento de uptime
RNF-03	Proteção de dados de geolocalização (LGPD)	Segurança	Atendida no desenho (RN-003, RN-023, RN-024); sem auditoria formal
RNF-04	Sessão por cookies httpOnly, Secure, SameSite=None; JWT validado no Gateway; rotas públicas restritas	Segurança	Atendido

ID	Requisito	Classificação	Situação
RNF-05	Validação cruzada do reporte colaborativo (3 confirmações)	Confiabilidade	Atendido (RN-015)
RNF-06	Não inventar dado: sem posição ou horário confiável, mostrar "sem previsão"	Confiabilidade	Atendido (RN-009)
RNF-07	Acessibilidade WCAG 2.2 AA e avaliação por heurísticas de Nielsen	Usabilidade	Contraste das cores calculado e coberto por teste; avaliação de Nielsen feita por um avaliador
RNF-08	Aplicativo web responsivo e instalável (PWA)	Portabilidade	Atendido
RNF-09	Tolerância a falha de integração externa (SEMOB, OSRM, geocodificação, CARTO)	Confiabilidade	Definida no Documento de Arquitetura, seção 10.3
RNF-10	Manutenibilidade: testes automatizados e CI	Manutenibilidade	1.792 testes automatizados passando no CI; resultados no Documento de Testes e Implementação
RNF-11	Frescor da posição: feed lido ao vivo com cache de 20 s, ETA atualizado sozinho	Eficiência de desempenho	Atendido

6. Casos de uso

Os diagramas de sequência (PlantUML) ficam em docs/diagramas/sequencia/. Os UC10 a UC30, UC115, UC171, UC172, UC173, UC190, UC195 e UC199 usam o número da US como identificador.

Caso de uso	US / RF	Diagrama
UC10 Realizar login com e-mail e senha	#10 / RF-02	UC10
UC11 Criar nova conta de usuário	#11 / RF-01	UC11
UC12 Realizar login com Google	#12 / RF-03	UC12
UC13 Recuperar senha	#13 / RF-04	UC13
UC14 Visualizar mapa com localização atual	#14 / RF-06	UC14
UC15 Buscar linha por número	#15 / RF-07	UC15
UC16 Rastrear posição do ônibus em tempo real	#16 / RF-08	UC16
UC17 Visualizar trajeto e paradas no mapa	#17 / RF-09	UC17
UC18 Ver detalhes de uma parada	#18 / RF-10	UC18
UC19 Ver tempo estimado de chegada	#19 / RF-11	UC19
UC20 Calcular rota de origem a destino	#20 / RF-12	UC20
UC21 Buscar parada por nome ou endereço	#21 / RF-13	UC21
UC22 Receber notificações de rotas preferidas	#22 / RF-22	UC22

Caso de uso	US / RF	Diagrama
UC23 Reportar ocorrência em uma linha	#23 / RF-18	UC23
UC24 Visualizar ocorrências reportadas	#24 / RF-19	UC24
UC25 Salvar rota favorita	#25 / RF-21	UC25
UC26 Confirmar ocorrência reportada	#26 / RF-20	UC26
UC27 Receber alerta de atraso na linha	#27 / RF-23	UC27
UC28 Receber alerta de cancelamento de viagem	#28 / RF-28	UC28
UC29 Gerenciar preferências de notificação	#29 / RF-24	UC29
UC30 Visualizar tutorial no primeiro acesso	#30 / RF-25	UC30
UC115 Planejar viagem por horário	#115 / RF-29	UC115
UC171 Pegar o ônibus	#171 / RF-15	UC171
UC172 Alerta de proximidade da parada	#172 / RF-16	UC172
UC173 ETA: mapeamento entre paradas	#173 / RF-11	UC173
UC190 Paradas de subir e descer na rota	#190 / RF-14	UC190
UC195 Alerta de proximidade com a tela bloqueada	#195 / RF-17	UC195
UC199 Fallback colaborativo de posição	#199 / RF-30	UC199

Sem diagrama de sequência: #31 (RF-05, descrito em gateway-auth-flow.md), #133 (RF-04), #136 (RF-26), #159 (RF-12), #174 (RF-10) e #201 (RF-27). A especificação textual de cada caso é o conjunto de cenários BDD de cada US, na seção 3.

7. Matriz de rastreabilidade

RF → US → caso de uso. A rastreabilidade completa, com PRs, casos de teste e arquivos de teste automatizado, está na matriz do Documento de Testes e Implementação.

RF	US	Caso de uso
RF-01	#11	UC11
RF-02	#10	UC10
RF-03	#12	UC12
RF-04	#13, #133	UC13
RF-05	#31	—
RF-06	#14	UC14
RF-07	#15	UC15
RF-08	#16	UC16
RF-09	#17	UC17
RF-10	#18, #174	UC18
RF-11	#19, #159, #173	UC19, UC173
RF-12	#20, #159	UC20
RF-13	#21	UC21
RF-14	#190	UC190
RF-15	#171	UC171
RF-16	#172	UC172

RF	US	Caso de uso
RF-17	#195	UC195
RF-18	#23	UC23
RF-19	#24	UC24
RF-20	#26	UC26
RF-21	#25	UC25
RF-22	#22	UC22
RF-23	#27	UC27
RF-24	#29	UC29
RF-25	#30	UC30
RF-26	#136	—
RF-27	#201	—
RF-28	#28	UC28
RF-29	#115	UC115
RF-30	#199	UC199

8. Restrições e premissas

Restrição ou premissa	Fonte
Sem PostGIS: geometrias em colunas JSON; a única junção espacial roda em Python na ingestão	ADR-02
Dados da SEMOB copiados por ingestão mensal em lote; os endpoints são públicos, sem documentação nem SLA, e o catálogo pode ficar até um mês desatualizado	ADR-03
Posição do ônibus lida ao vivo (cache de 20 s) e nunca guardada; sem histórico de posições	ADR-04
Sem ambiente de homologação separado: `homolog` publica na produção da Vercel; os quatro apps do Fly.io e o banco Neon são compartilhados, por custo	ADR-19
Escala a zero no Fly: partida a frio na primeira requisição após ociosidade	ADR-12
Gateway como ponto único de entrada; cookies `SameSite=None` + `Secure`, porque frontend (Vercel) e Gateway (Fly) são cross-site	ADR-07
Integrações externas atrás de contrato `typing.Protocol` (mock e real)	ADR-06
Posição colaborativa só em memória do processo: com mais de uma máquina no Fly.io, o fallback pode não aparecer para quem consulta em outra máquina; trocar por armazenamento compartilhado é uma mudança só de implementação	ADR-25
Horário planejado é estimado (saída do terminal mais tempo de percurso proporcional); feriados não são considerados	ADR-26
Web Push exige chaves VAPID no servidor; no iPhone, só com o app instalado	ADR-16
Escopo geográfico: projeto-piloto em Taguatinga e Ceilândia (os dados da SEMOB cobrem o DF inteiro)	Documento de Visão
Sem validação em campo: raio de embarque, tolerância do	Documento de Testes e Implementação

Restrição ou premissa	Fonte
ETA e regra "ônibus chegando" não foram medidos com ônibus real	

9. Declaração de uso ético e responsável de inteligência artificial

O projeto adota um fluxo de trabalho com agentes de IA generativa no desenvolvimento e na documentação (metodologia "Agentic Workflow"; regras de operação em docs/boas-praticas-ia.md). Todo o código foi gerado por agentes de IA, conduzidos pelos seis integrantes do grupo, que especificaram, revisaram e aprovaram. Este DRS foi levantado a partir de issues, PRs, código e documentos do repositório, com apoio de um assistente de IA. Ferramentas: Claude Code (modelos Claude Sonnet, Opus e Haiku, da Anthropic) e OpenCode (modelos Qwen, DeepSeek e Big Pickle, além dos do Claude). Período: de 15/08 a 30/10/2026.

- **Supervisão humana:** todo conteúdo gerado é revisado por membros do grupo, que permanecem responsáveis pela exatidão e pela conformidade acadêmica. Este documento foi revisado por Davi Eduardo Costa Miranda (orquestrador).
- **Checagem de confiabilidade:** cada afirmação deste DRS aponta para uma fonte verificável (issue, PR, arquivo de código ou documento de testes).
- **Privacidade e sigilo:** nenhum dado de usuário, credencial ou dado pessoal entra neste documento; os casos de teste usam contas de teste.
- **Integridade acadêmica:** a contribuição da ferramenta é declarada aqui, conforme a orientação do template.

10. Histórico de revisões

Versão	Data	Alteração
1.0	30/10/2026	Versão entregue, criada a partir das issues [US], dos PRs mergeados, do código de homolog e da documentação de testes
1.1	31/10/2026	Inclusão de RF-28 (alerta de cancelamento, #28), RF-29 (planejamento por horário, #115) e RF-30 (posição colaborativa, #199), com RN-023 a RN-028

Protótipo de interface de usuário

O protótipo de alta fidelidade não possui documento próprio e **não foi feito no Figma**. Ele está no repositório do projeto, na pasta `prototipo-alta-fid/` (20 telas em HTML), e publicado em:

https://davieduardo001.github.io/tcc-ciencia-computacao-ucb/

As telas estão reproduzidas nas páginas seguintes.

Protótipo de Alta Fidelidade — Movecity

Protótipo navegável (HTML, CSS e JavaScript, sem backend) das telas do backlog — issue #33

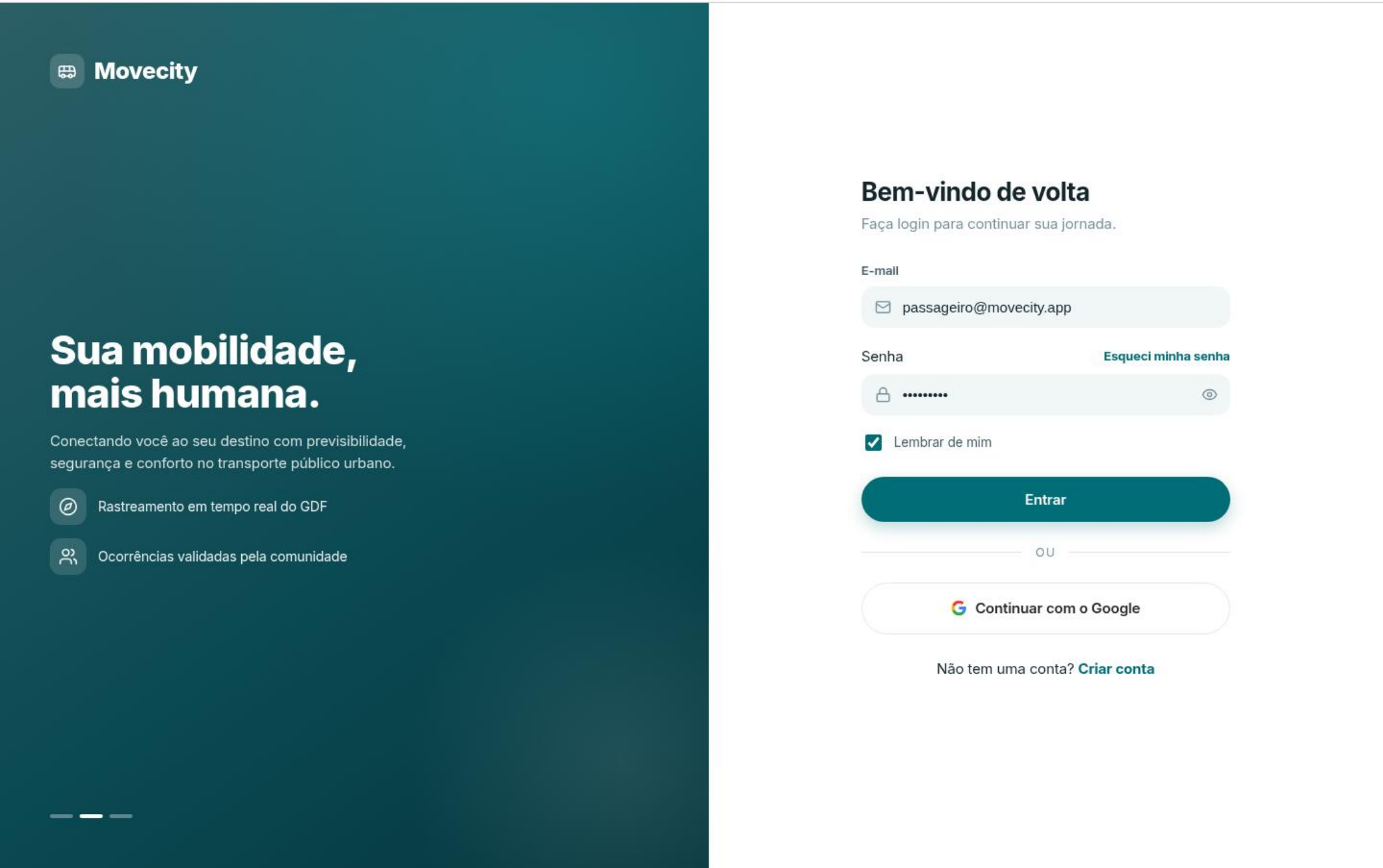
Código-fonte no repositório do projeto:

https://github.com/davieduardo001/tcc-ciencia-computacao-ucb/tree/main/prototipo-alta-fid/web

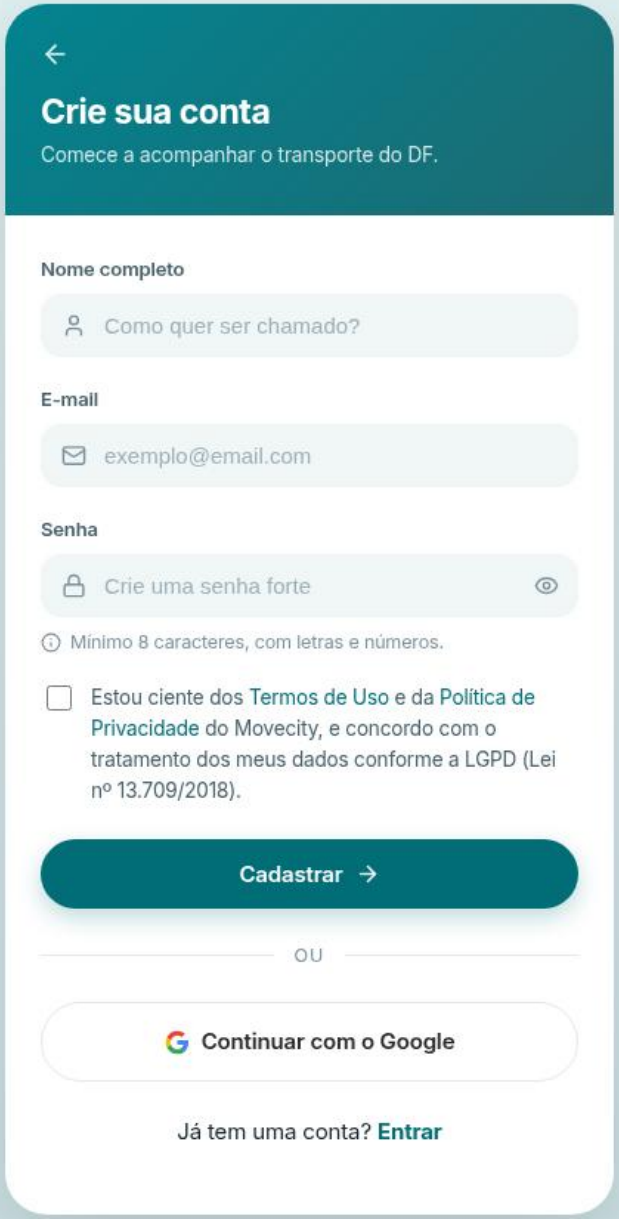
Para abrir: baixar a pasta e abrir index.html no navegador (funciona sem servidor).

As páginas seguintes mostram as 19 telas do fluxo. O mapa é ilustrativo (imagem estática).

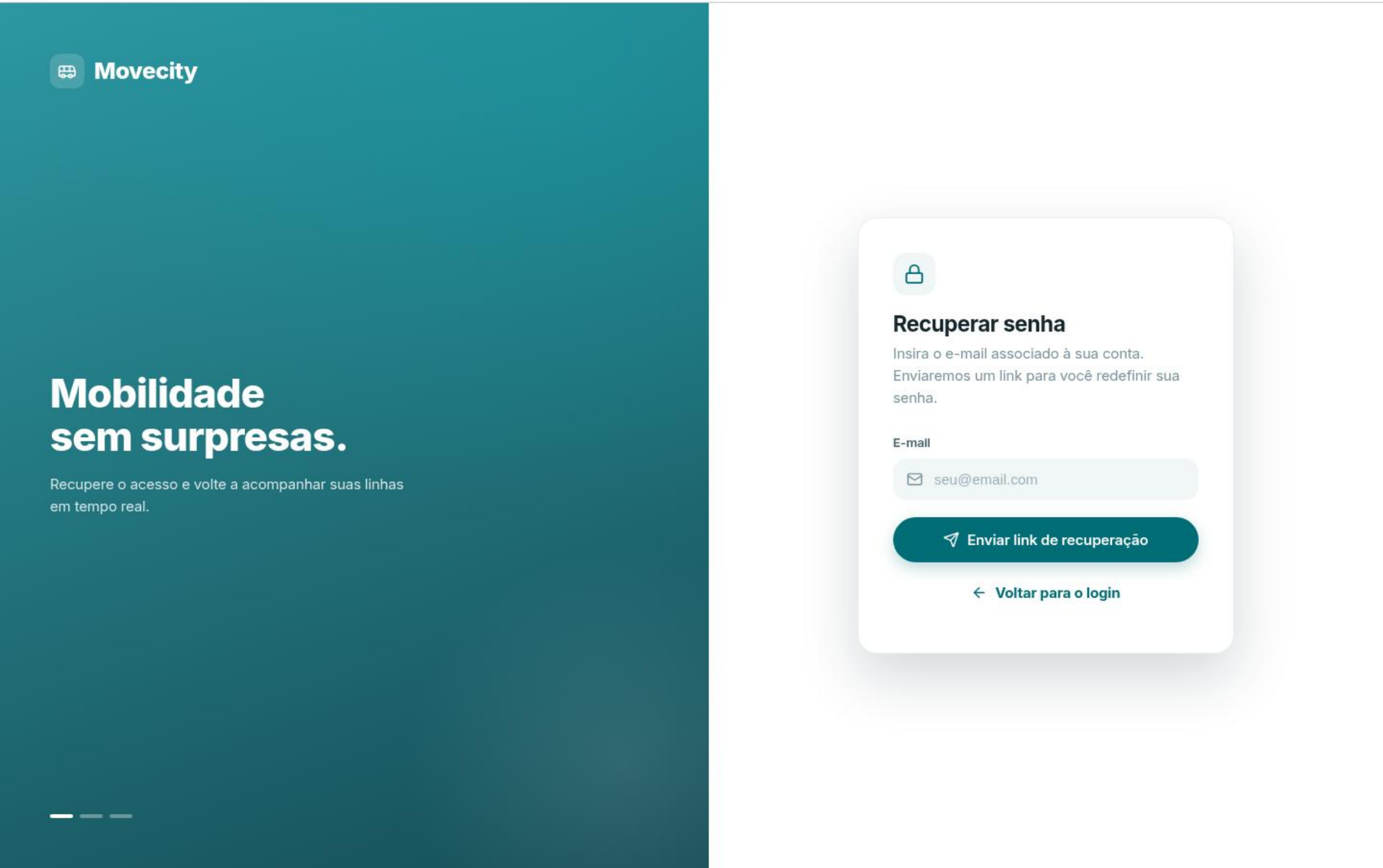
01. Login (US #10, #12)



02. Criar conta (US #11)



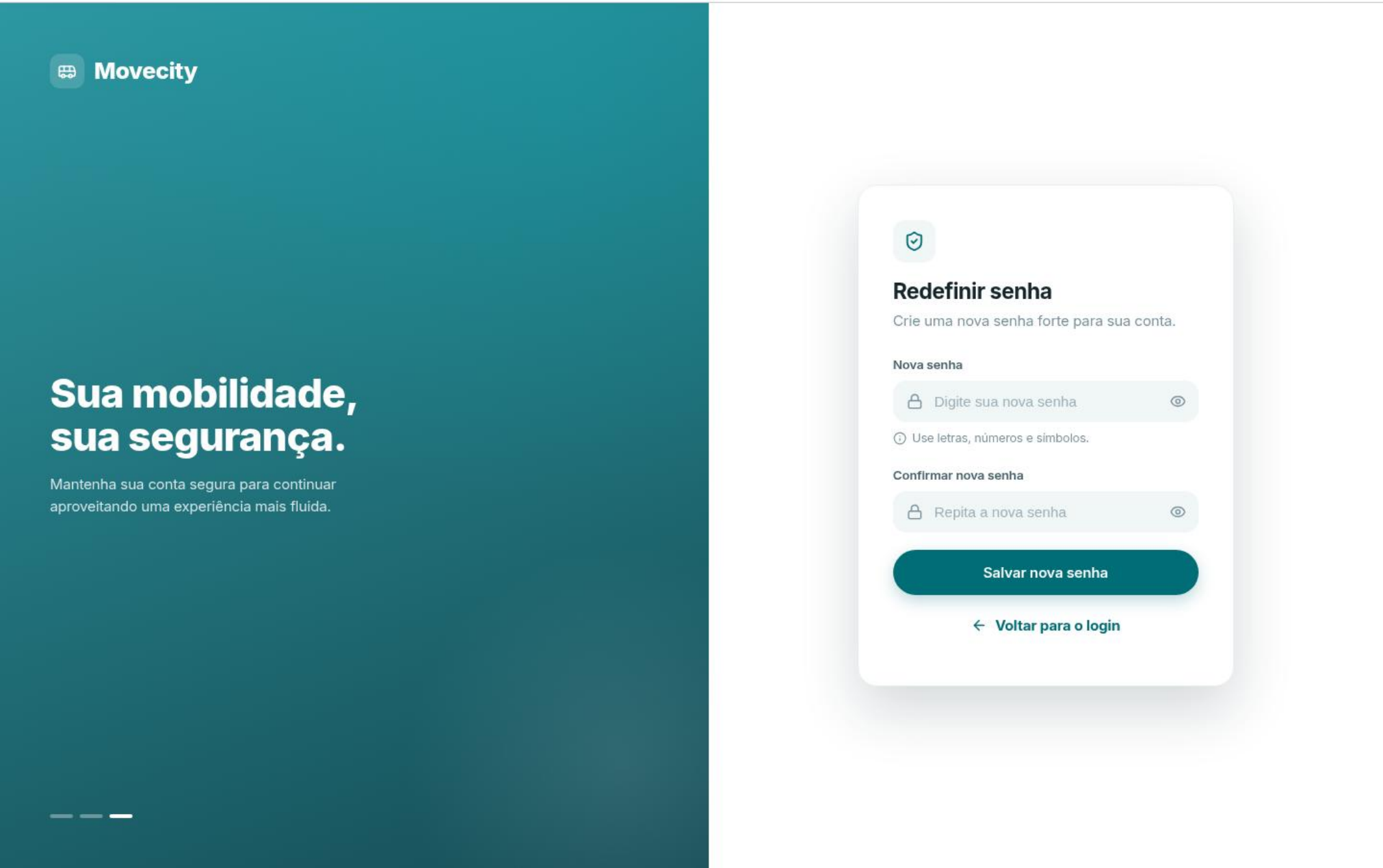
03. Recuperar senha (US #13)



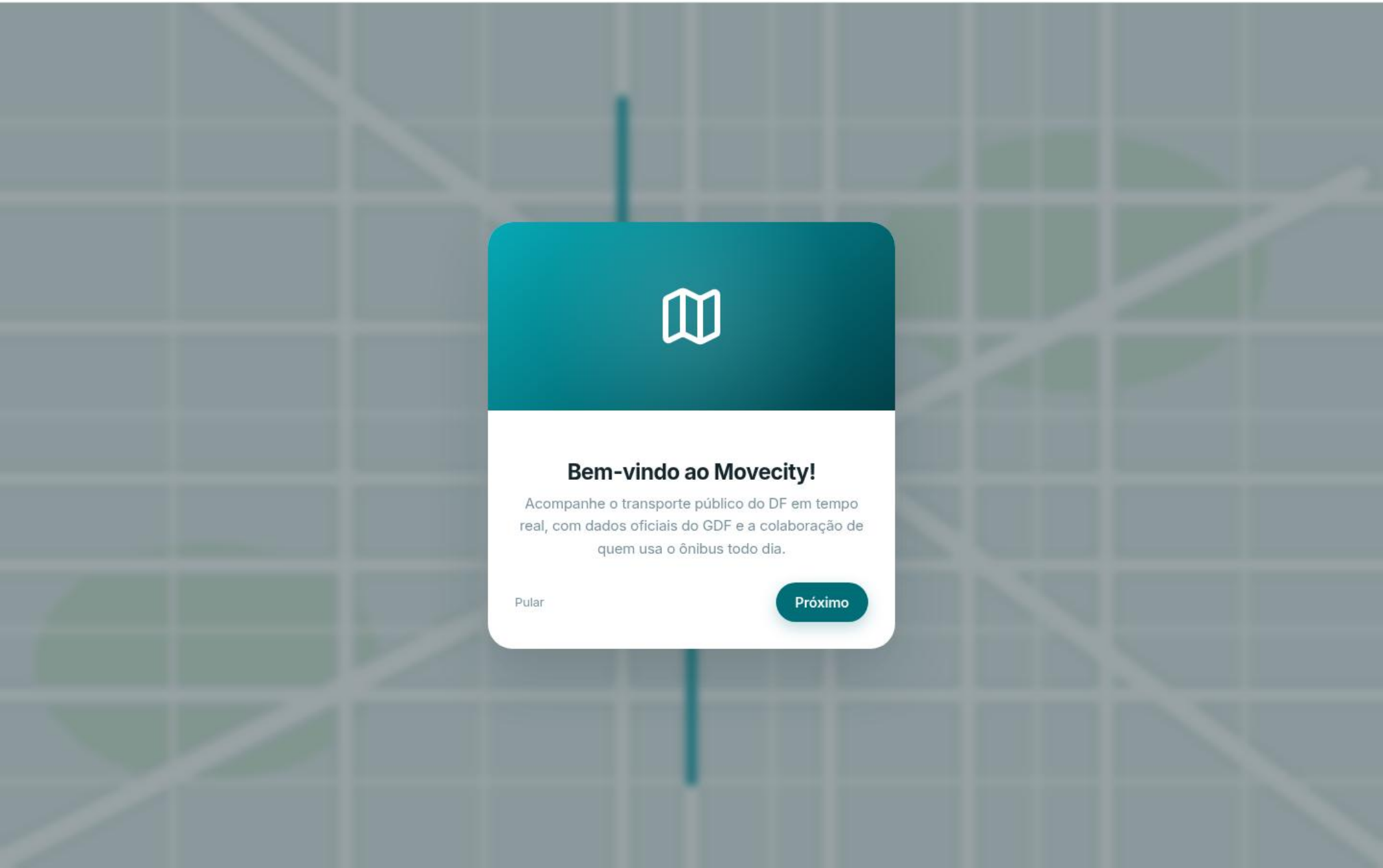
04. Link enviado (US #13)



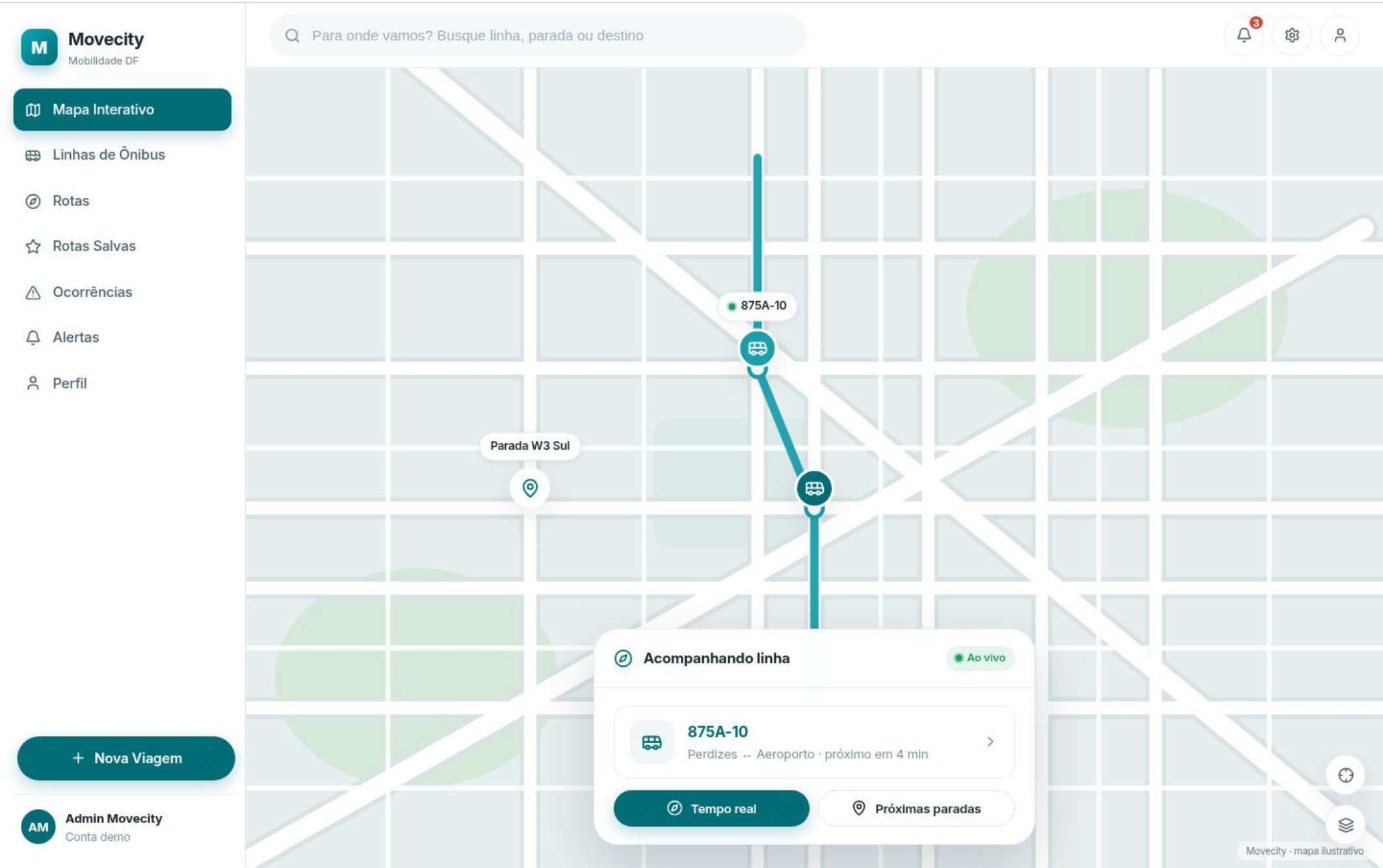
05. Redefinir senha (US #13)



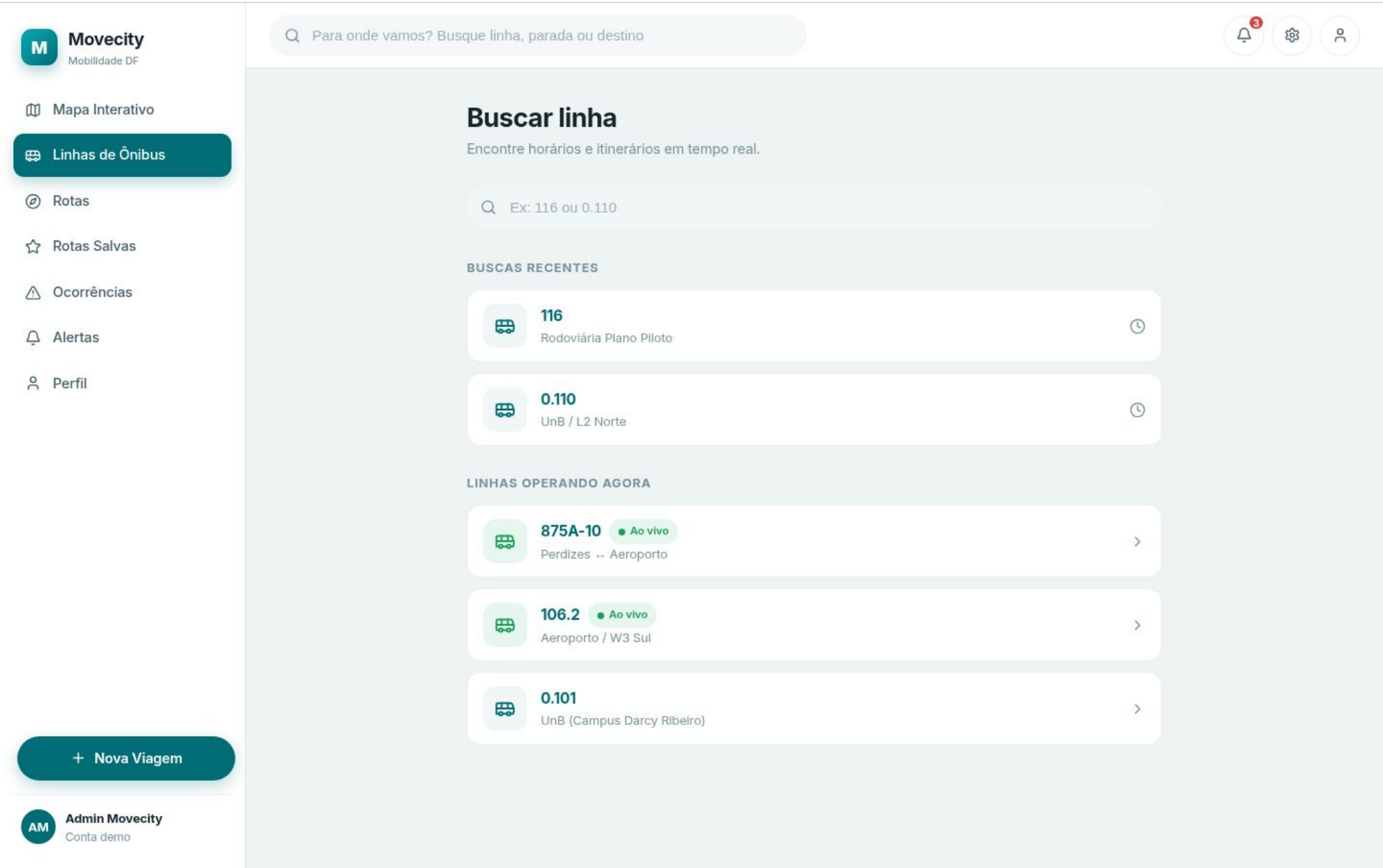
06. Tutorial / onboarding (US #30)



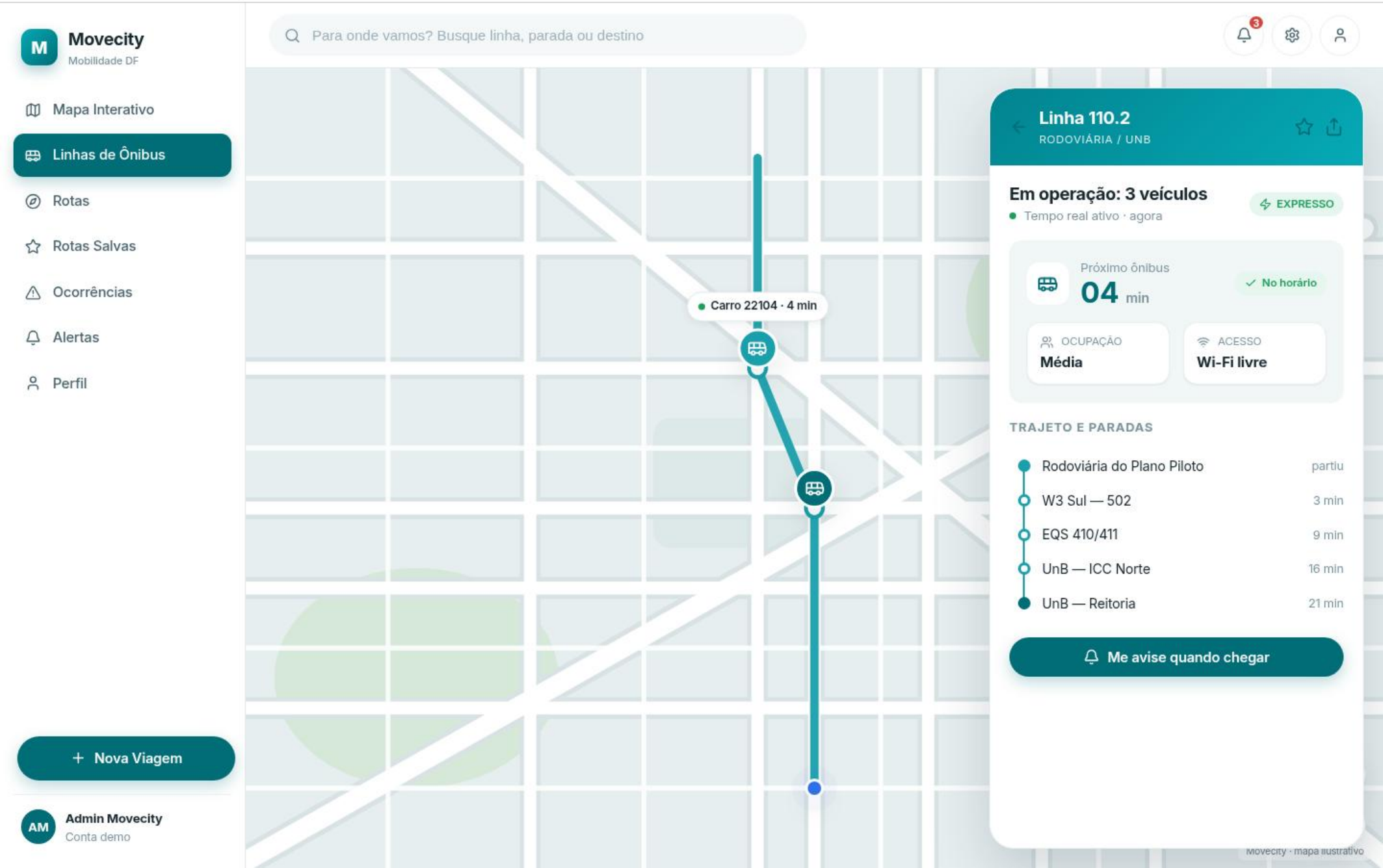
07. Mapa e localização atual (US #14)



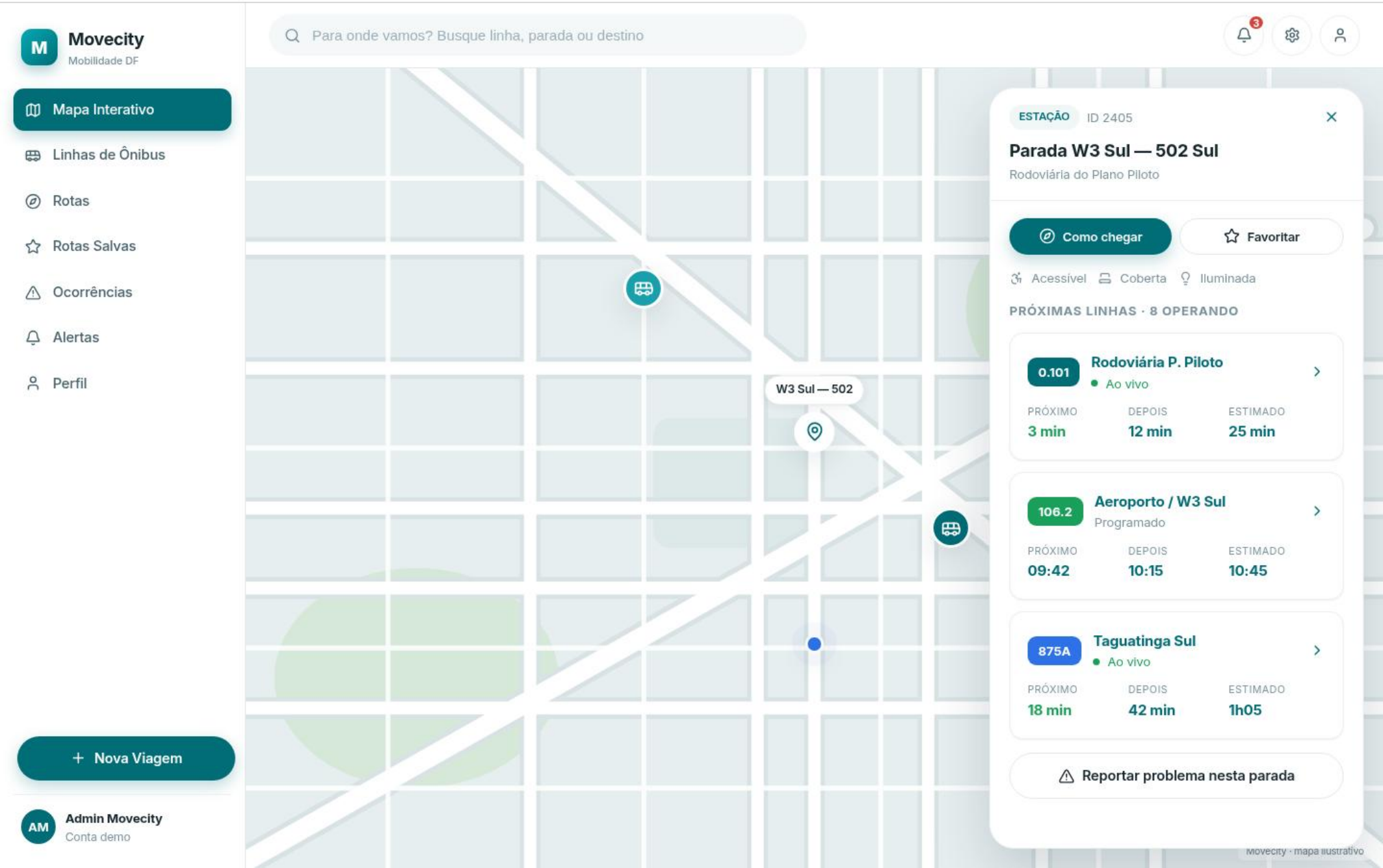
08. Buscar linha por número (US #15)



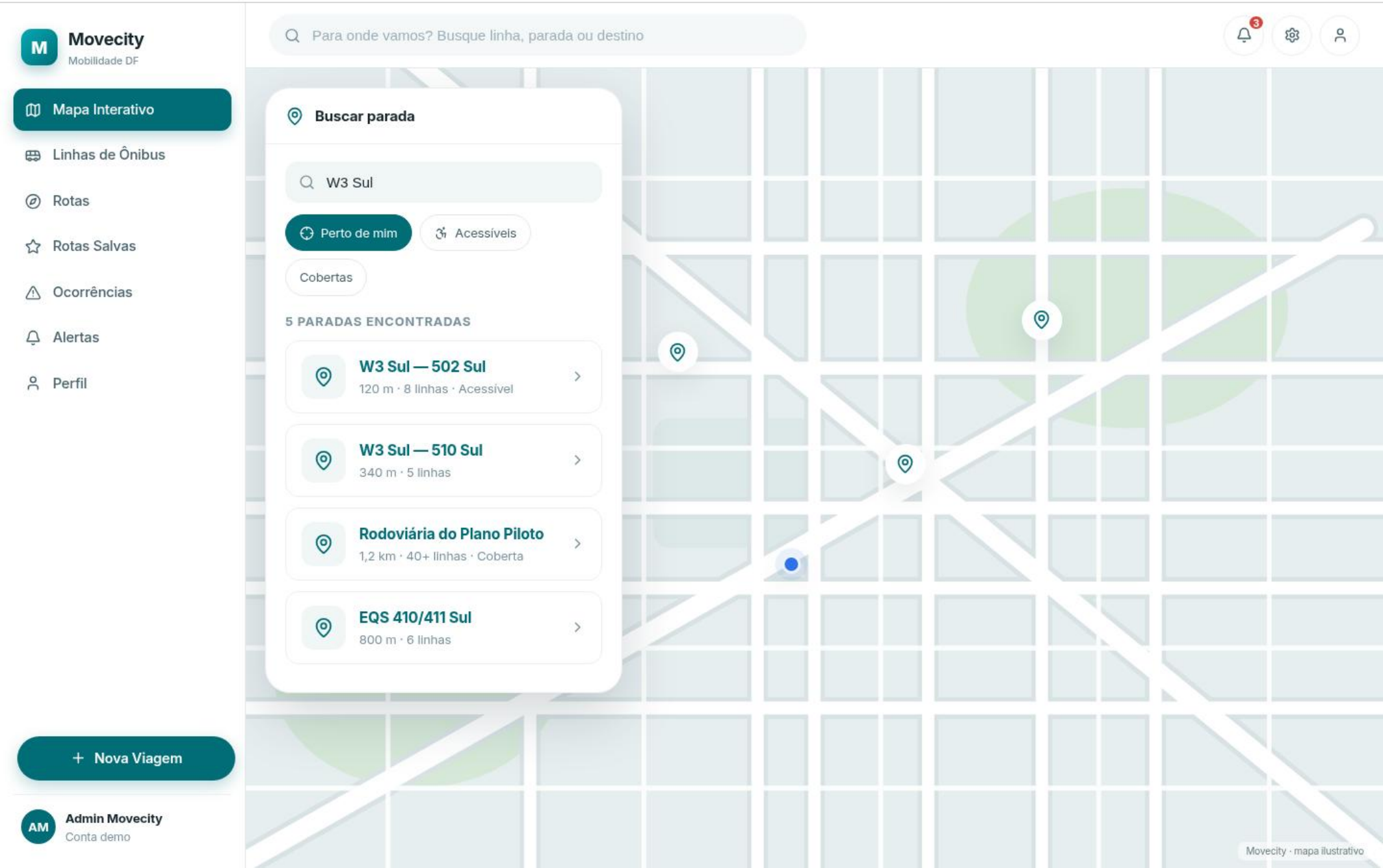
09. Linha em tempo real (US #16, #17, #19)



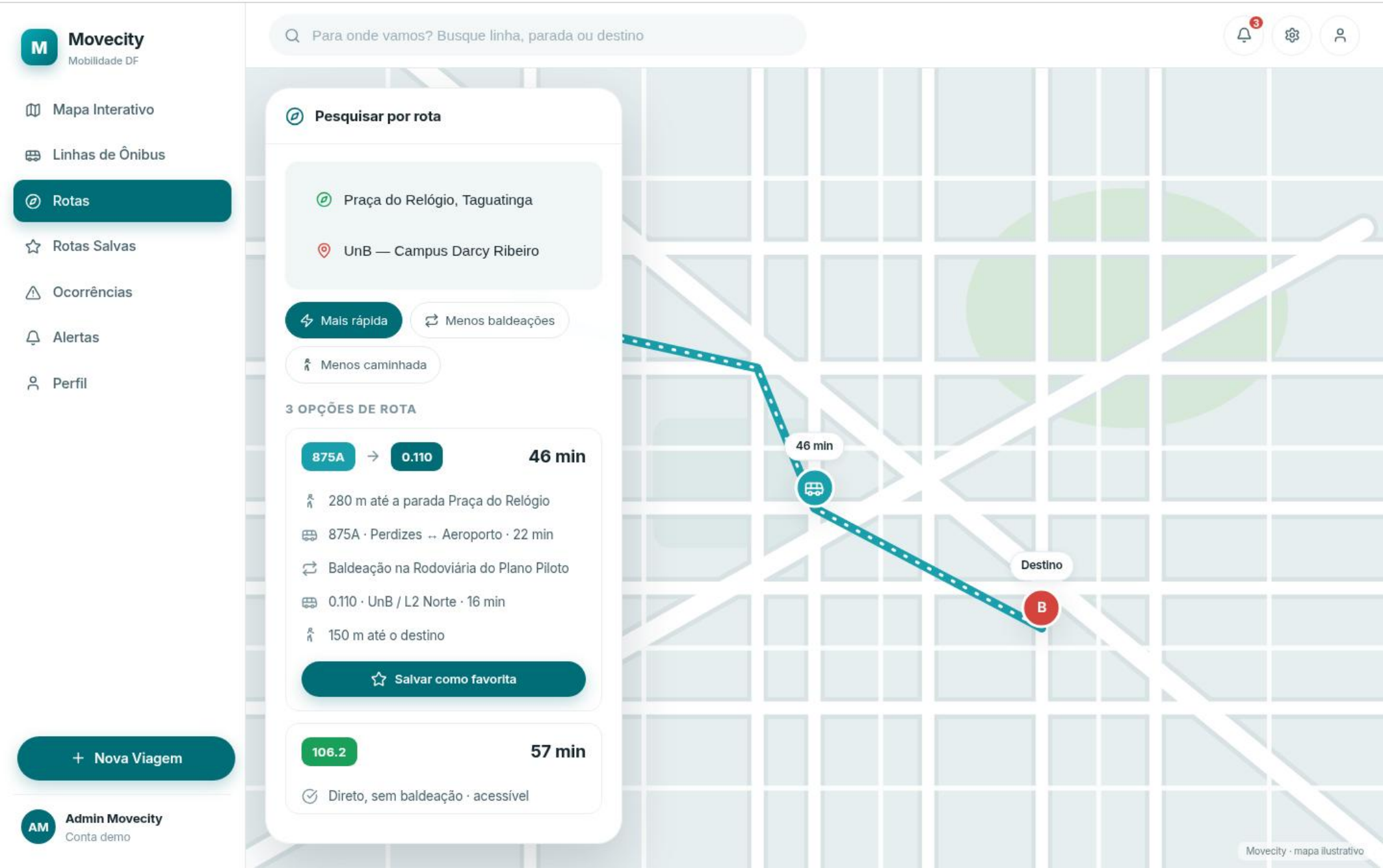
10. Detalhes da parada (US #18, #19)



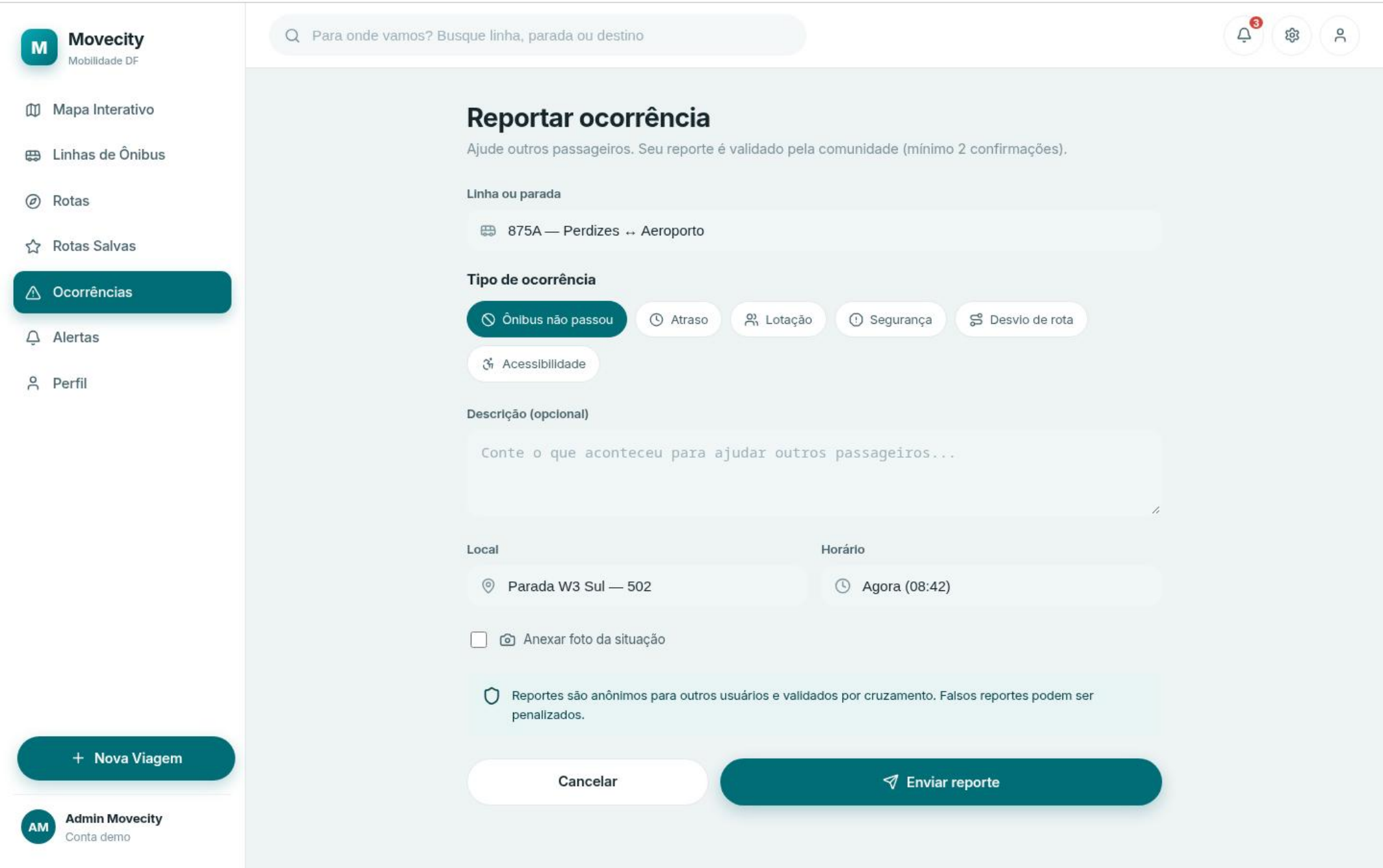
11. Buscar parada por nome ou endereço (US #21)



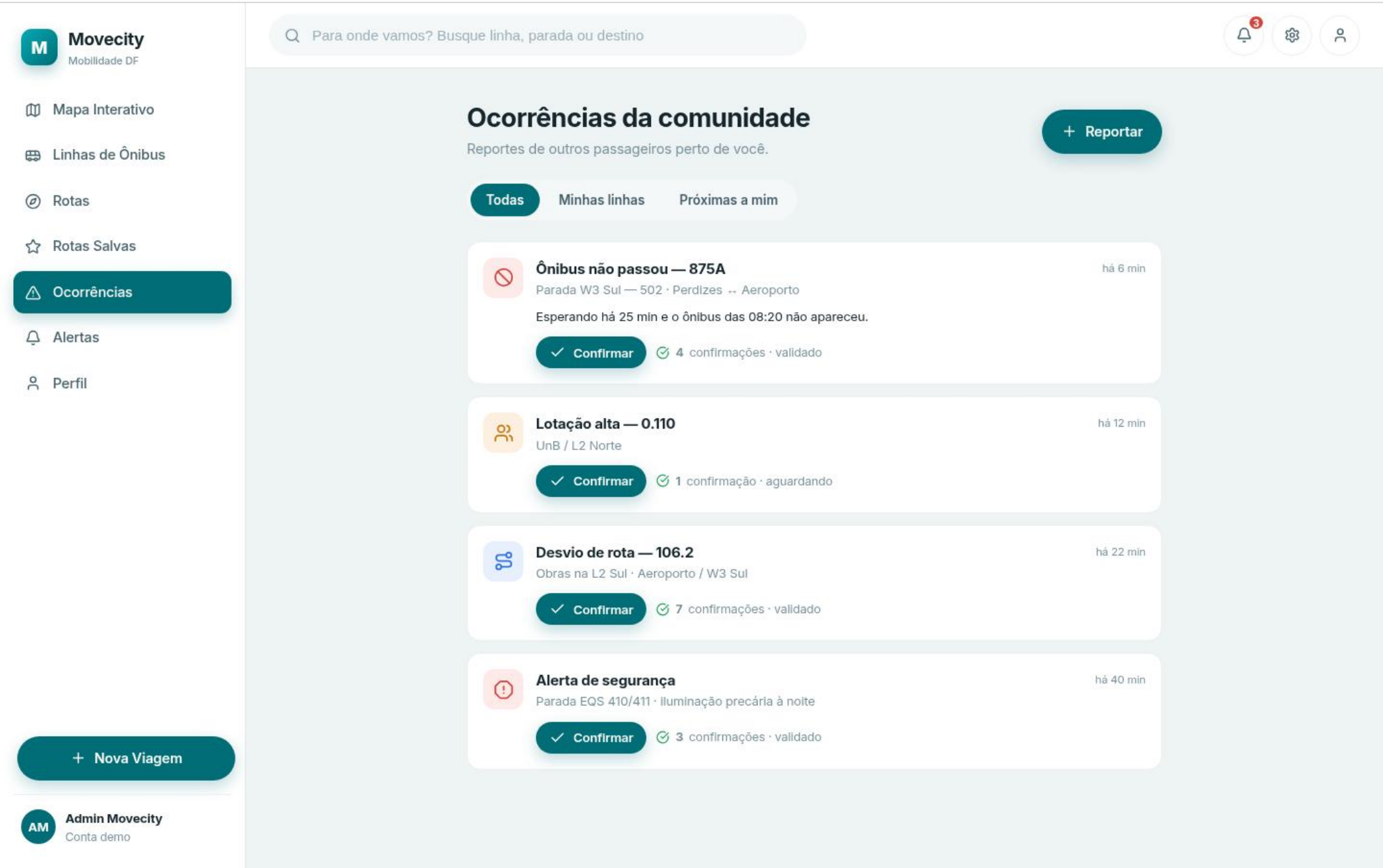
12. Calcular rota (US #20)



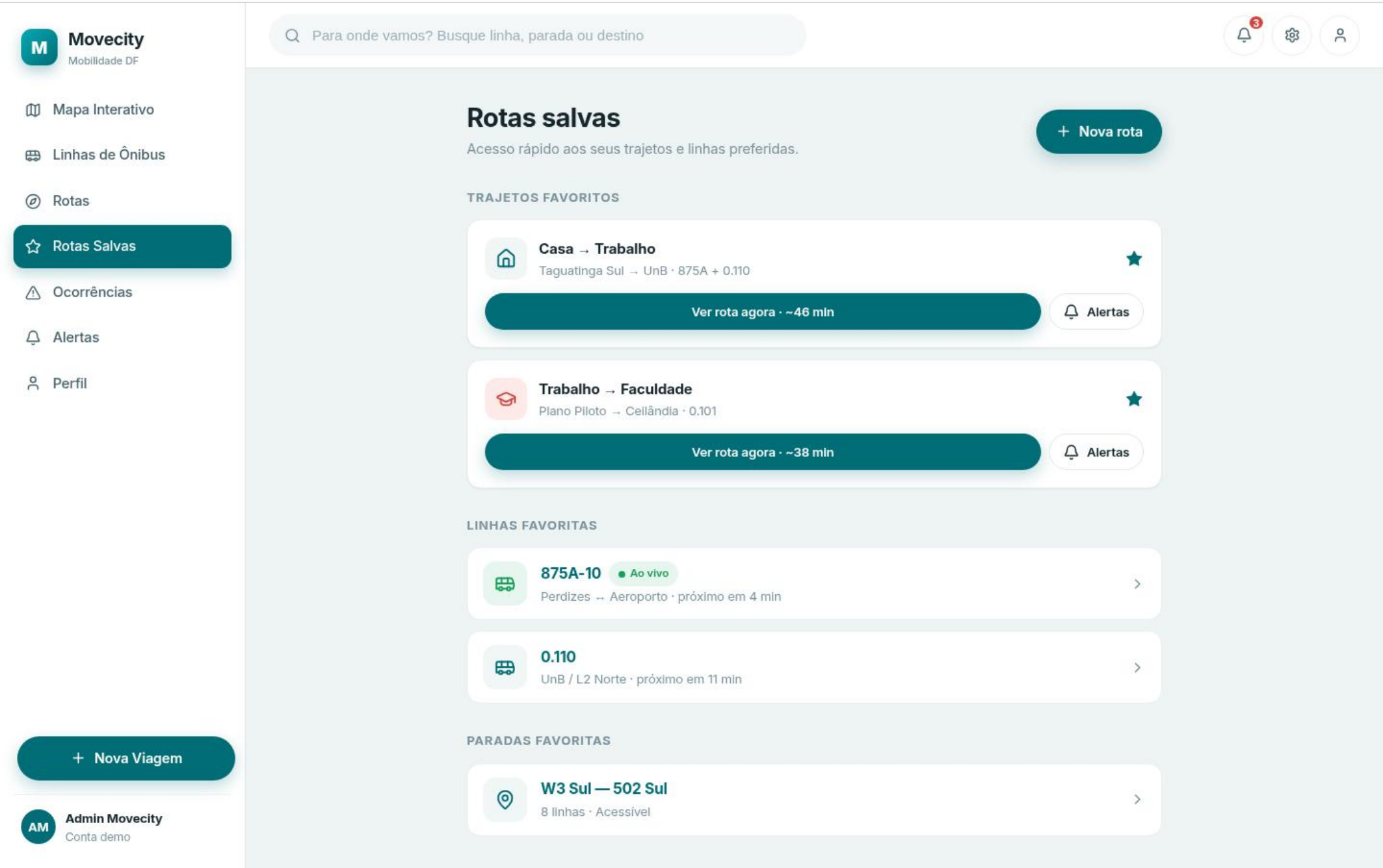
13. Reportar ocorrência (US #23)



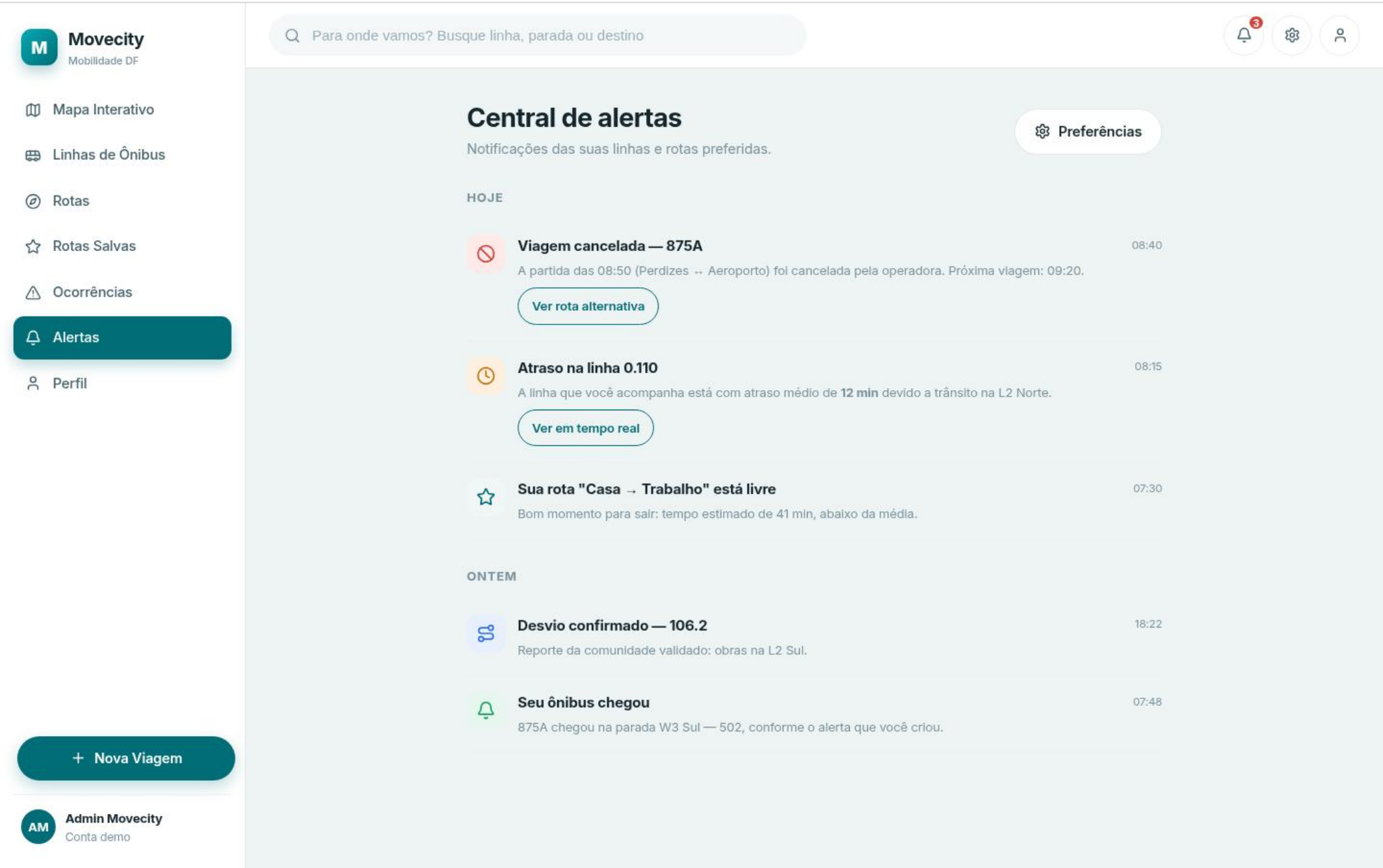
14. Ocorrências da comunidade (US #24, #26)



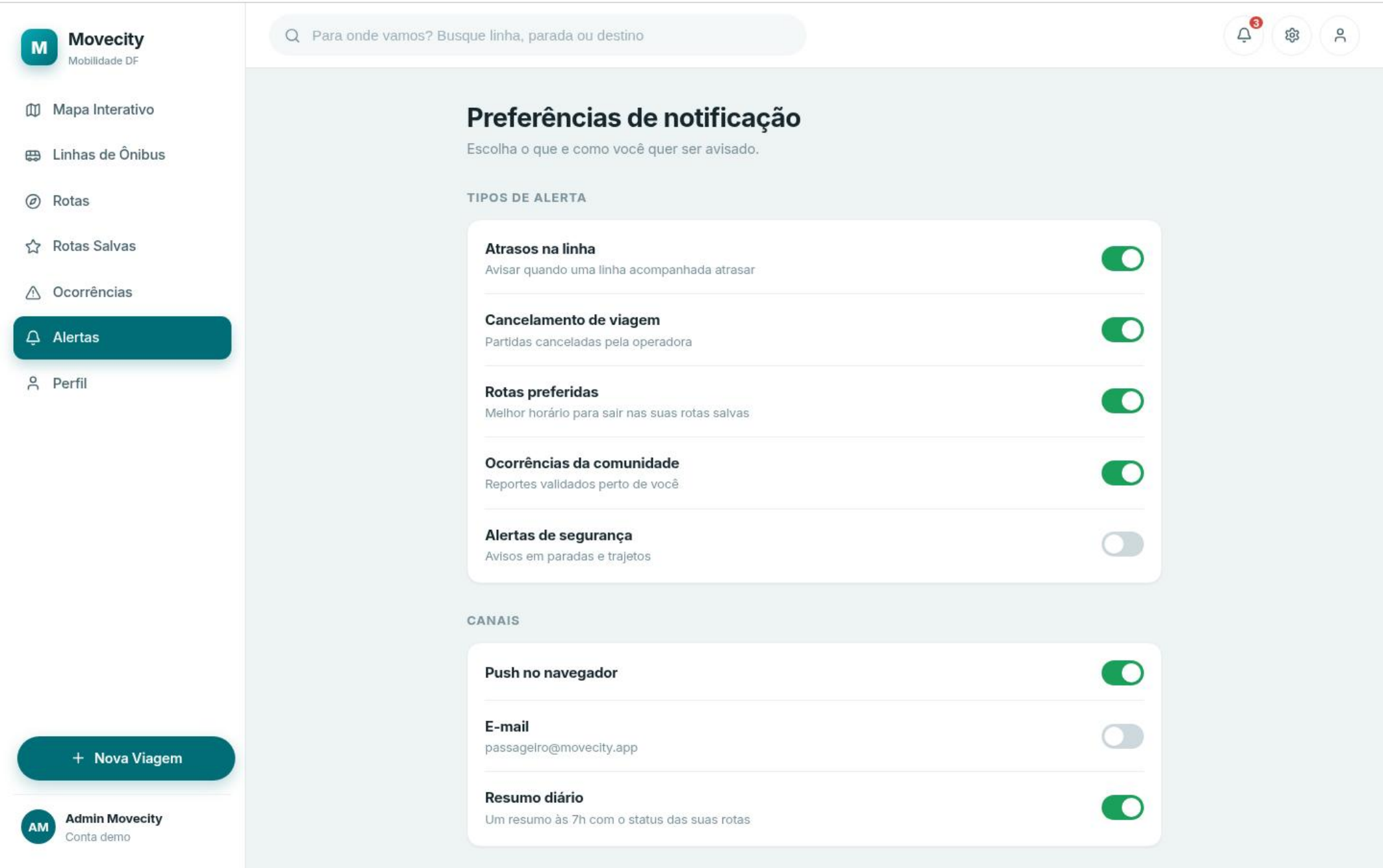
15. Rotas favoritas (US #25)



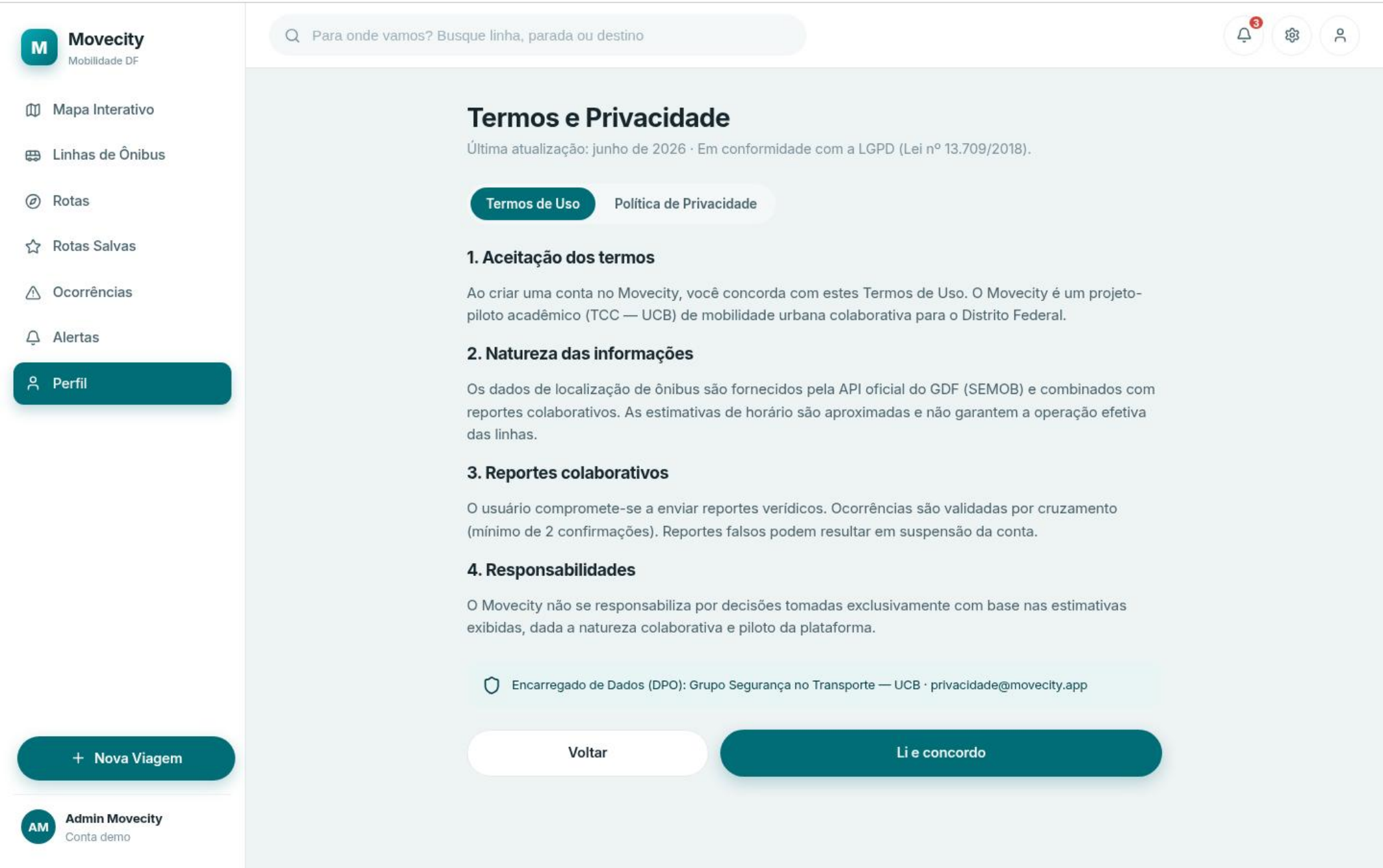
16. Central de alertas (US #22, #27, #28)



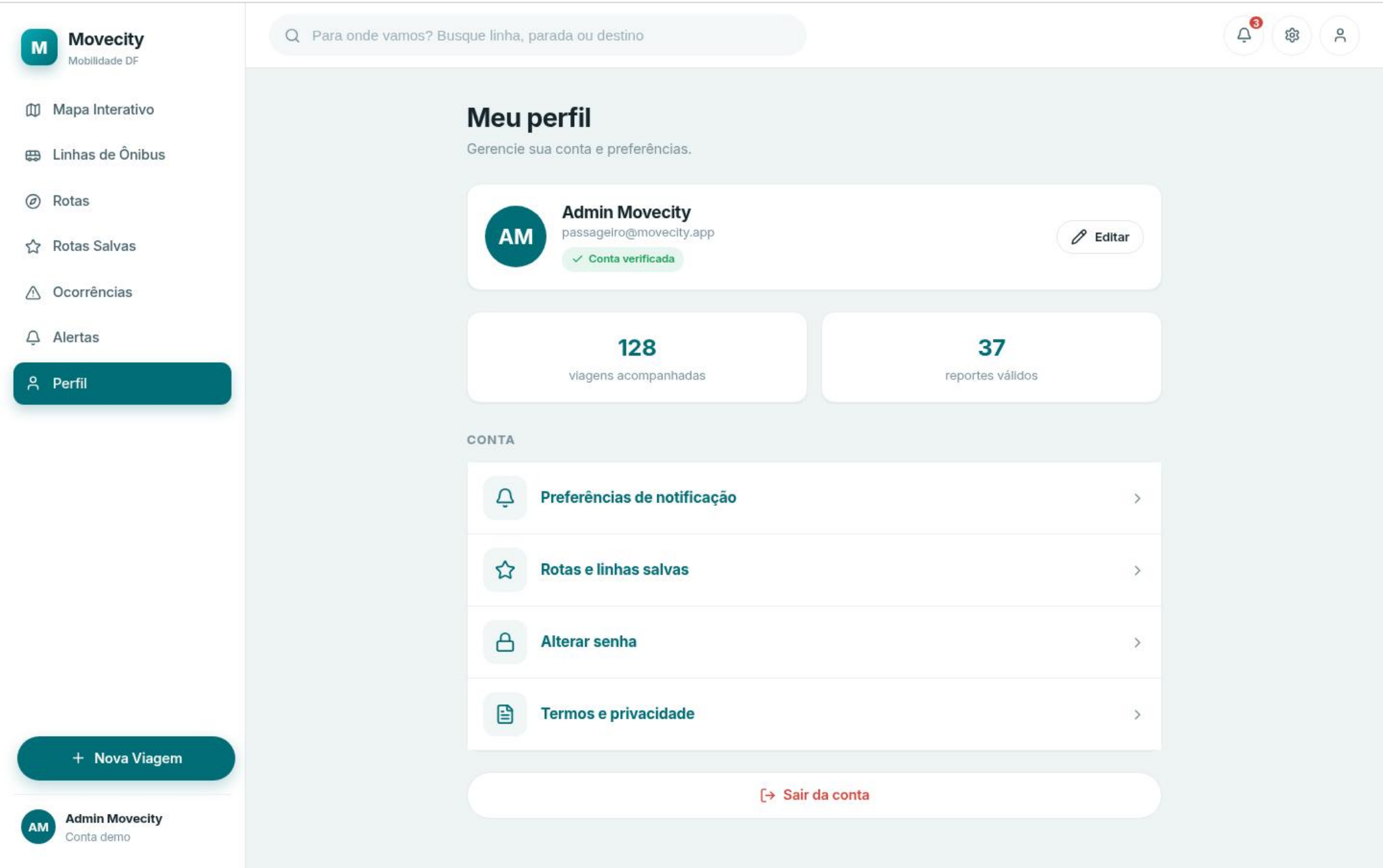
17. Preferências de notificação (US #29)



18. Termos e privacidade (US #32)



19. Perfil e conta (US #31)



Grupo Segurança no Transporte (UCB)

Movecity
Documento de Arquitetura de Software

Versão 3.2

Histórico da Revisão

Data	Versão	Descrição	Autor
12/jun/26	1.0	Versão inicial do documento	Grupo Segurança no Transporte
18/set/26	2.0	Atualização pós-implementação: correção das seções divergentes do código (persistência, processos, implementação, implantação e dados) e inclusão da seção 10 — Evolução Arquitetural	Grupo Segurança no Transporte
28/out/26	3.0	Revisão contra o código de homolog (ambiente publicado): correção de afirmações que o código contradiz (regiões do Fly, validação cruzada, retenção de ocorrências, worker de colaboração, papel do Gateway na renovação da sessão, acesso ao banco), atualização das tabelas, endpoints, workflows e requisitos não funcionais com o que foi verificado e o que não foi medido, e registro das ADR-16 a ADR-24	Grupo Segurança no Transporte
30/out/26	3.1	Versão entregue: revisão do texto, remoção de repetições e atualização dos números de testes	Grupo Segurança no Transporte
31/out/26	3.2	Inclusão do alerta de cancelamento (#28), do planejamento por horário (#115) e da posição colaborativa (#199): ADR-25 a ADR-27, processo 8, tabelas de dados e números de testes	Grupo Segurança no Transporte

Índice Analítico

Introdução 4
Finalidade 4
Escopo 4
Definições, Acrônimos e Abreviações 4
Restrições e Requisitos Arquiteturais 5
Situação dos requisitos 7
Visão de Casos de Uso 7
Visão Lógica 8
Representação do Domínio da Aplicação 8
Decisões Arquiteturais 9
Representação da Arquitetura Lógica 11
Representação do Funcionamento da Arquitetura 13
Visão de Processos 13
Visão da Implementação 15
Visão de Implantação 16
Visão de Dados 18
Volume e Desempenho 20
Evolução Arquitetural (junho a outubro de 2026) 21
Registro de decisões (ADR-01 a ADR-15) 21
Decisões da Sprint 3 e da Sprint 4 (ADR-16 a ADR-27) 24
Degradação prevista por dependência externa 27
Limites conhecidos da arquitetura atual 28
Divergências entre a v1.0 e a implementação 28
Referências 29

Documento de Arquitetura de Software

Introdução

A introdução deste Documento de Arquitetura de Software apresenta uma visão geral da estrutura e das decisões técnicas adotadas no desenvolvimento do Movecity. Está organizada conforme o modelo "4+1" de Philippe Kruchten (1994), que descreve a arquitetura de software por meio de cinco visões complementares: Lógica, de Processos, de Implementação, de Implantação e de Casos de Uso. Este modelo permite que diferentes públicos — analistas, desenvolvedores, operadores e stakeholders — compreendam o sistema a partir de perspectivas distintas e complementares.

Finalidade

Este documento oferece uma visão geral arquitetural abrangente do sistema Movecity, usando diversas visões arquiteturais para representar diferentes aspectos do sistema. O objetivo é capturar e comunicar as decisões arquiteturais significativas tomadas pelo Grupo Segurança no Transporte durante a especificação do projeto. O público-alvo inclui os desenvolvedores do grupo, o orientador acadêmico e avaliadores da UCB. O documento adota a estrutura "4+1" de Kruchten (1994) como referencial metodológico.

Escopo

O Movecity é um aplicativo web de mobilidade urbana colaborativa voltado para o Distrito Federal, com projeto-piloto nas cidades-satélite de Ceilândia e Taguatinga. A plataforma combina dados GPS oficiais da frota do STPC/DF (via API da SEMOB/GDF) com reportes colaborativos dos próprios passageiros para mitigar o fenômeno do "ônibus fantasma" e reduzir a insegurança nas paradas de ônibus. Este documento descreve a arquitetura de software que suporta essas funcionalidades, **como ela está implementada no ambiente publicado (homolog) em 28/10/2026**. Onde o planejamento original (v1.0) divergiu do código, o texto descreve o código e a divergência está registrada na seção 10. O que não foi entregue ou não foi medido aparece como tal, nunca como se existisse.

Definições, Acrônimos e Abreviações

Termo	Definição
DF	Distrito Federal
STPC/DF	Sistema de Transporte Público Coletivo do Distrito Federal
SEMOB	Secretaria de Mobilidade do GDF
GDF	Governo do Distrito Federal
API	Application Programming Interface — interface de comunicação entre sistemas
GPS	Global Positioning System — sistema de posicionamento global
GeoJSON	Formato de dados geoespaciais baseado em JSON
SSR	Server-Side Rendering — renderização do lado do servidor
SSG	Static Site Generation — geração de site estático
JWT	JSON Web Token — padrão de autenticação stateless
ORM	Object-Relational Mapper — mapeador objeto-relacional
CI/CD	Continuous Integration / Continuous Delivery — integração e entrega contínua
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018)
PostGIS	Extensão geoespacial para PostgreSQL. Avaliada na v1.0 e **não adotada** — ver seção 10
OSRM	Open Source Routing Machine — serviço de roteamento sobre a malha viária do OpenStreetMap
CARTO	Provedor de tiles de mapa usado como base visual do Leaflet
ETA	Estimated Time of Arrival — tempo estimado de chegada
ADR	Architecture Decision Record — registro de decisão

Termo	Definição
	arquitetural
UC	Caso de Uso (Use Case)
US	User Story — história de usuário
MVC	Model-View-Controller — padrão arquitetural
UCB	Universidade Católica de Brasília
TCC	Trabalho de Conclusão de Curso
PWA	Progressive Web App — aplicativo web instalável, com manifesto e *service worker*
Web Push	Notificação entregue ao navegador pelo serviço de push do fabricante, mesmo com o app fechado
VAPID	Voluntary Application Server Identification — par de chaves que identifica o servidor perante os serviços de Web Push
SMTP	Simple Mail Transfer Protocol — protocolo de envio de e-mail
SameSite	Atributo de cookie que define se o navegador o envia em requisições entre sites diferentes

Restrições e Requisitos Arquiteturais

Esta seção descreve os requisitos de negócio e de usuário com impacto sobre os atributos de qualidade da arquitetura, bem como as restrições do projeto. Os atributos de qualidade estão em conformidade com a norma NBR ISO/IEC 25010. A Tabela 1 traz o requisito e a solução arquitetural adotada; a Tabela 1.1 diz, para cada requisito, o que foi de fato verificado e o que **não foi medido**.

Atributo de Qualidade	Requisito de Arquitetura	Solução
Desempenho	O carregamento inicial do mapa e das posições dos ônibus deve ocorrer em no máximo 3 segundos em conexões 4G.	Posições de GPS lidas ao vivo da origem, com cache em memória de 20 segundos compartilhado entre todos os usuários (seção 5); catálogo de linhas, trajetos e paradas servido do banco, sem consultar a SEMOB a cada busca; trajeto cacheado junto com a linha.
Interoperabilidade	O sistema deve consumir dados GPS no formato GeoJSON provido pela API oficial da SEMOB/GDF sem necessidade de transformação proprietária.	Backend FastAPI com módulo de ingestão (semob_source.py, ingestao_semob.py); endpoints RESTful com documentação automática via OpenAPI.
Usabilidade	A interface deve ser responsiva e acessível para usuários com diferentes níveis de letramento digital, priorizando dispositivos móveis.	Interface em Next.js (App Router) pensada para o celular (folha inferior no mapa), instalável como PWA (manifest.ts e public/sw.js); componente de mapa via Leaflet JS com interação por toque.
Confiabilidade	Reportes colaborativos só devem ser confirmados quando validados por outros usuários independentes.	Validação cruzada no serviço de Colaboração: ocorrência de tipo sujeito a validação nasce pendente e vira confirmada ao receber 3 confirmações de usuários diferentes do autor (Ocorrencia.LIMITE_CONFIRMACOES); a restrição de uma confirmação por usuário é garantida por UNIQUE (ocorrencia_id, usuario_id) no

Atributo de Qualidade	Requisito de Arquitetura	Solução
		banco. Acidente e segurança são exibidos de imediato, sem esperar confirmação. Anti-spam: no máximo 3 reportes por usuário por hora (LIMITE_REPORTES_POR_HORA).
Segurança	Dados de geolocalização dos usuários devem ser tratados em conformidade com a LGPD. Sessões autenticadas devem ser gerenciadas de forma segura.	Autenticação por JWT (HS256) enviado em cookies httpOnly; o Gateway valida o token localmente, sem round-trip ao Auth Service; senhas com hash bcrypt; bloqueio da conta por 15 min após 5 senhas erradas; a viagem do passageiro não guarda posição (ADR-15); a query string é omitida do log de acesso do Gateway e da Mobilidade (shared/logs.py); a posição do usuário não vai para a URL (ADR-24); HTTPS forçado nos apps do Fly (force_https = true) e na Vercel. Limites conhecidos na seção 10.4.
Facilidade de manutenção	O código deve ser modular e permitir a evolução por serviço sem reestruturação arquitetural.	Separação de responsabilidades por camadas (frontend, gateway, serviços de domínio, dados); serviços independentes, cada um com seu Dockerfile, fly.toml e requirements.txt; testes automatizados executados pelo GitHub Actions a cada PR. O catálogo ingerido cobre o DF inteiro (923 linhas), não só Ceilândia e Taguatinga.
Portabilidade	O backend deve ser implantável em diferentes provedores de nuvem sem dependência de serviços proprietários.	Imagens Docker (python:3.12-slim); banco PostgreSQL padrão (Neon serverless); nenhuma dependência de SDK de nuvem proprietário no código da aplicação.
Escalabilidade	O sistema deve suportar crescimento de usuários sem redesenho arquitetural, especialmente nos horários de pico (5h–9h e 17h–20h).	Serviços independentes atrás de um Gateway, escaláveis por app no Fly.io; PostgreSQL serverless no Neon; frontend servido pela Vercel. **Limite:** o cache de posições, o limite de tentativas de recuperação de senha e a deduplicação de alertas vivem na memória de cada processo e não são compartilhados entre instâncias; não houve teste de carga.
Disponibilidade	O sistema deve ter disponibilidade mínima de 99% nos horários de pico definidos.	Cada serviço roda em uma única região (gru, São Paulo); o pipeline reprova o deploy se GET /health não responder 200 (seção 7). Não há monitoramento contínuo nem alerta de indisponibilidade configurados no repositório, e as máquinas escalam a zero (ADR-12). A meta de 99% **não foi medida**.

Tabela 1 – Restrições e requisitos arquiteturais

Situação dos requisitos

Resultados detalhados no Documento de Testes e Implementação. Aqui fica o que foi verificado e o que **não foi medido**.

Requisito	O que foi verificado	O que não foi medido
Desempenho (3 s em 4G)	O cache de 20 s do feed está no código.	Tempo de carregamento do mapa em 4G, latência do Gateway e da SEMOB, partida a frio dos apps do Fly.
Precisão do tempo de chegada (ETA)	Lógica coberta por testes automatizados e por casos manuais em produção.	Tolerância do ETA em campo (proposta: 2 min para ETA até 10 min) e a regra "ônibus chegando" (400 m e 2 min) com um ônibus real.
Disponibilidade (99% no pico)	Deploys com teste de /health no pipeline.	Disponibilidade real: não há série de medição nem monitoramento contínuo.
Confiabilidade (validação cruzada)	Regra de 3 confirmações implementada e testada.	Comportamento com muitos usuários reais.
Segurança e LGPD	Sessão renovada em produção sem novo login; coordenadas do usuário fora da URL; viagem sem coluna de posição.	Revisão jurídica da LGPD.
Usabilidade	Avaliação heurística de Nielsen por um avaliador; teste automatizado de contraste.	Teste com usuários reais; leitor de tela, tamanho de toque e conexão lenta.
Manutenção (testes)	1.792 testes automatizados passando no CI (952 de backend, 840 de frontend); cobertura de 93,7% (backend) e 89,1% (frontend), medida em 16/10.	Cobertura no CI.
Integração entre as peças	49 casos manuais executados em produção: 32 aprovados, 11 aprovados em parte, 1 reprovado por configuração, 3 não verificáveis fora de campo e 2 pendentes.	Teste automatizado de ponta a ponta entre frontend, Gateway e serviços.
Escalabilidade	—	Teste de carga.

Tabela 1.1 – Situação verificada dos requisitos

Visão de Casos de Uso

O Movecity foi especificado por meio de histórias de usuário (User Stories), organizadas por épicos funcionais. Cada item está vinculado à sua respectiva issue no GitHub, onde os comentários contêm os artefatos completos de especificação: o protótipo de alta fidelidade (prototipo-alta-fid/), diagramas de sequência UML e critérios de aceite BDD.

Épico: Autenticação e Conta

- #11 — Criar Nova Conta de Usuário
- #10 — Realizar Login com E-mail e Senha
- #12 — Login via Provedor Social (Google)
- #13 — Recuperação de Senha (Esqueci minha senha)
- #31 — Gerenciar Ciclo de Vida da Sessão (logout e renovação de token)

Épico: Exploração do Mapa

- #14 — Visualizar Mapa com Localização Atual
- #15 — Buscar Linha por Número
- #16 — Rastrear Posição do Ônibus em Tempo Real

- #17 — Visualizar Trajeto e Paradas da Linha no Mapa
- #18 — Ver Detalhes de uma Parada
- #19 — Ver Tempo Estimado de Chegada do Ônibus até Minha Parada
- #20 — Calcular Rota de Origem até Destino por Ônibus
- #21 — Buscar Parada por Nome ou Endereço

Épico: Colaboração e Reportes

- #22 — Receber Notificações de Rotas Preferidas
- #23 — Reportar Ocorrência em uma Linha
- #24 — Visualizar Ocorrências Reportadas por Outros Passageiros
- #25 — Salvar Rota Favorita
- #26 — Confirmar Ocorrência Reportada por Outro Passageiro
- #27 — Receber Alerta de Atraso na Linha Acompanhada
- #29 — Gerenciar Preferências de Notificação

Épico: Onboarding

- #30 — Visualizar Tutorial no Primeiro Acesso

Visão Lógica

Esta seção descreve os componentes significativos da arquitetura do Movecity, sua organização em camadas e pacotes, as principais classes do domínio e as decisões técnicas que definem o design do sistema.

Representação do Domínio da Aplicação

O diagrama de classes da UML é uma representação estática da estrutura do sistema, mostrando as entidades do domínio, seus atributos, operações e os relacionamentos entre elas. Para o Movecity, o diagrama de classes representa as principais entidades de negócio e como elas se relacionam para sustentar as funcionalidades da plataforma.

Principais entidades do domínio:

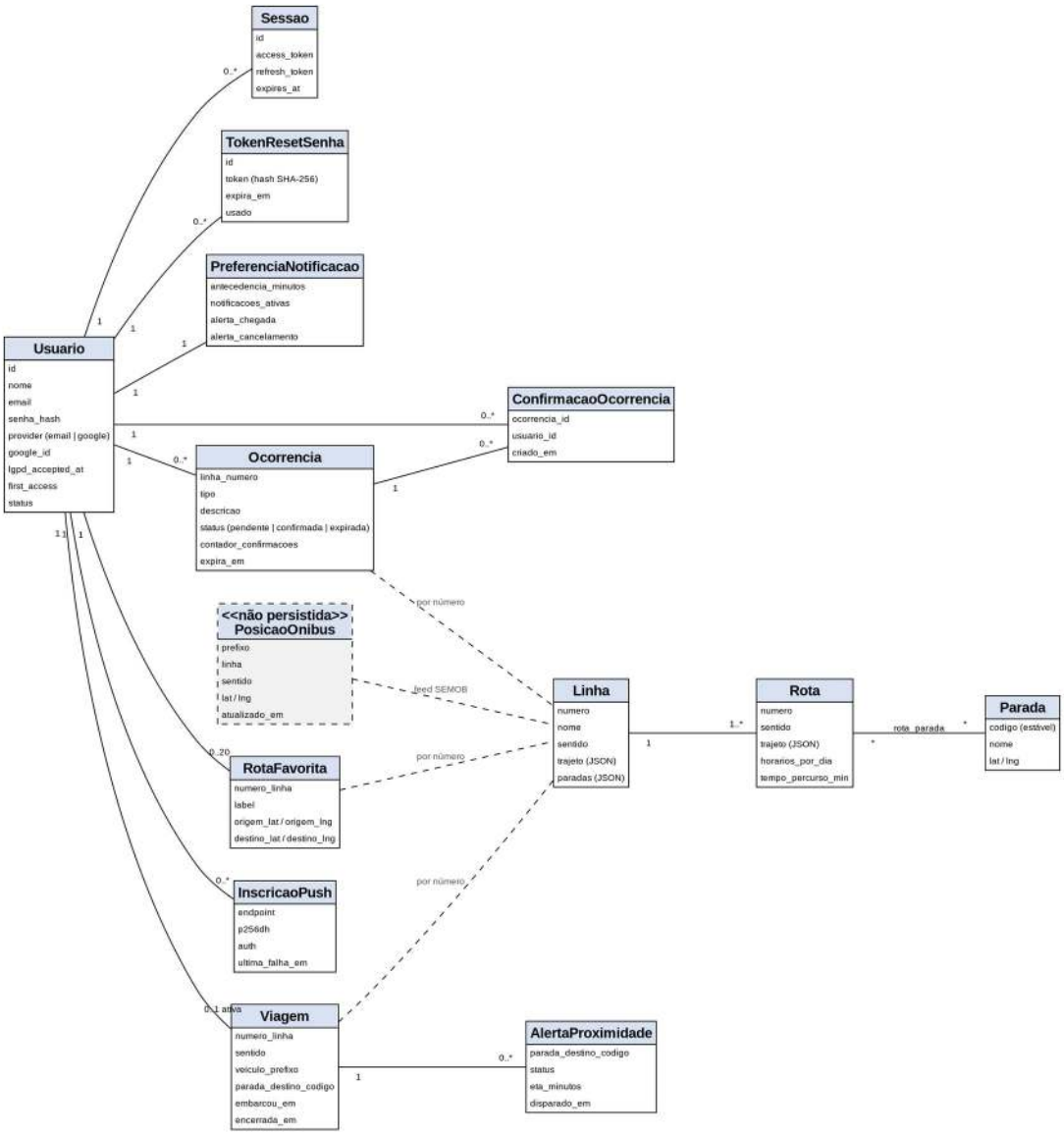


Figura 1 — Diagrama de classes simplificado do domínio do Movecity

Atributos principais, conferidos com os modelos de homolog. As tabelas e colunas completas estão na seção 8.

Decisões Arquiteturais

Esta subseção documenta as principais decisões técnicas tomadas para o Movecity, com suas justificativas.

Item Arquitetural	Decisão	Justificativa
Estilo arquitetural	Microserviços com API Gateway (padrão Gateway API)	Permite escalabilidade independente por serviço, isolamento de falhas e evolução incremental da plataforma sem redesenho total.
Linguagem — Frontend	TypeScript (v5.x)	Tipagem estática reduz erros em tempo de desenvolvimento; suporte nativo no Next.js e vasto ecossistema.

Item Arquitetural	Decisão	Justificativa
Framework — Frontend	Next.js 14.2.5 (React 18, App Router)	Roteamento baseado em arquivos simplifica a estrutura; integração nativa com a Vercel, onde o frontend é publicado.
Linguagem — Backend	Python 3.12	Equipe familiarizada com a linguagem; ecossistema maduro para o tratamento dos dados geoespaciais da SEMOB. Na prática, a junção espacial foi implementada com a biblioteca padrão (sem Shapely), o que manteve a imagem Docker enxuta.
Framework — Backend	FastAPI (>= 0.115)	Framework assíncrono de alto desempenho; documentação automática via OpenAPI 3.0; validação com Pydantic.
Componente de mapa	Leaflet JS (v1.9) com base CARTO Voyager e recuo automático para o tile padrão do OpenStreetMap	Leaflet é gratuito, open source e suporta GeoJSON nativamente. A base do CARTO tem contraste mais baixo que o tile padrão do OSM, deixando legível o que importa na tela (linha, parada, veículo); passou a exigir chave, que é gratuita (5 milhões de tiles/mês). Sem a chave configurada, o app recua para o tile padrão do OSM — nenhuma tela quebra por falta de credencial.
Formas de persistência	Banco relacional para o domínio; cache em memória de processo para o feed de posições; cache em tabela para o catálogo de linhas	Três naturezas de dado, três estratégias: entidades do domínio são relacionais; posição de GPS muda a cada poucos segundos e não vale persistir (cache em memória, TTL de 20 s); dados de linha são estáticos e caros de obter (cache na tabela linha, populado em lote).
Banco de dados	PostgreSQL (Neon serverless), **sem PostGIS**	Free tier generoso e escalabilidade automática. A extensão PostGIS foi avaliada na v1.0 e não adotada: as consultas geoespaciais do produto se resumem a uma junção por proximidade entre traçado e abrigos de parada, resolvida em Python com índice de grade na ingestão (seção 10, ADR-02). Geometrias são guardadas em colunas JSON.
Serviços de terceiros	SEMOB/GDF (dados.semob.df.gov.br): /espaciais, /horario, /pontos e /posicao; OSRM (router.project-osrm.org); Nominatim/OpenStreetMap (busca e geocodificação reversa de lugares); Google Routes API (opcional, só com GOOGLE_MAPS_API_KEY); CARTO Basemaps; Google OAuth 2.0; Gmail SMTP; serviços de Web Push dos navegadores	Dados oficiais das frotas e da rede, sem chave e sem billing — são os mesmos endpoints que alimentam o app oficial "DF no Ponto". OSRM ajusta o traçado à malha viária real; Nominatim resolve endereços do planejador de rotas. Google OAuth reduz o atrito no cadastro. Gmail SMTP entrega o e-mail de recuperação de senha (ADR-20). Web Push entrega o alerta de proximidade com a tela bloqueada (ADR-16).
Integração com fontes externas	Padrão *provider*: contrato declarado	Permite desenvolver e testar sem rede e

Item Arquitetural	Decisão	Justificativa
	como typing.Protocol em providers/contratos.py, com implementação mock e implementação real	sem cota de API, e trocar a fonte em um único ponto (o service), sem reescrever rota HTTP nem regra de negócio. Foi o que permitiu sair do mock para os dados reais do SEMOB sem tocar no LinhaService.
Dependências	Um requirements.txt por serviço, além do combinado na raiz	Cada imagem Docker instala apenas o que o serviço usa, reduzindo o tamanho da imagem. Ver a armadilha correspondente na seção 10 (ADR-08).
Deploy — Backend	Fly.io — **quatro aplicações independentes** (movecity-gateway, movecity-auth, movecity-mobilidade, movecity-colaboracao), cada uma com seu Dockerfile e fly.toml	Deploy simples via Dockerfile e escala independente por serviço. Separar as aplicações mantém a promessa de isolamento de falhas do estilo de microserviços: a queda de um serviço não derruba os demais.
Deploy — Frontend	Vercel	Integração nativa com Next.js e deploy automático por push. homolog publica em --prod no domínio público da Vercel (movecity-frontend.vercel.app), usado como ambiente de homologação e para as demonstrações; a main não tem deploy próprio de frontend (ADR-19).
CI/CD	GitHub Actions	Integração nativa com o repositório; pipelines de teste e deploy automatizados sem custo adicional.
Autenticação	JWT (HS256) em cookies httpOnly — access token de 1 h e refresh token de 7 dias; sessões registradas na tabela sessoes	Os cookies httpOnly não são lidos por JavaScript; o Gateway valida o access token localmente, sem round-trip; o refresh token é rotacionado a cada renovação e deixa de valer no logout. A renovação é disparada pelo frontend ao receber 401 (ADR-21).
Notificações	Web Push com chaves VAPID (pywebpush), enviado pelo serviço de Mobilidade	Entrega o aviso de proximidade com o app fechado. Desligado quando as chaves não estão configuradas (ADR-16).
E-mail	SMTP do Gmail (smtplib, porta 587, STARTTLS), enviado pelo serviço de autenticação	Sem dependência nova; usado só na recuperação de senha (ADR-20).
Aplicativo instalável	PWA: manifest.ts e public/sw.js	O *service worker* guarda só a casca do aplicativo e recebe os Web Push; dado de transporte nunca é cacheado.

Tabela 2 – Decisões arquiteturais

Representação da Arquitetura Lógica

O diagrama de pacotes da UML representa a organização lógica do sistema em camadas e seus relacionamentos de dependência. O Movecity segue uma arquitetura de microserviços com API Gateway, organizada nas seguintes camadas:

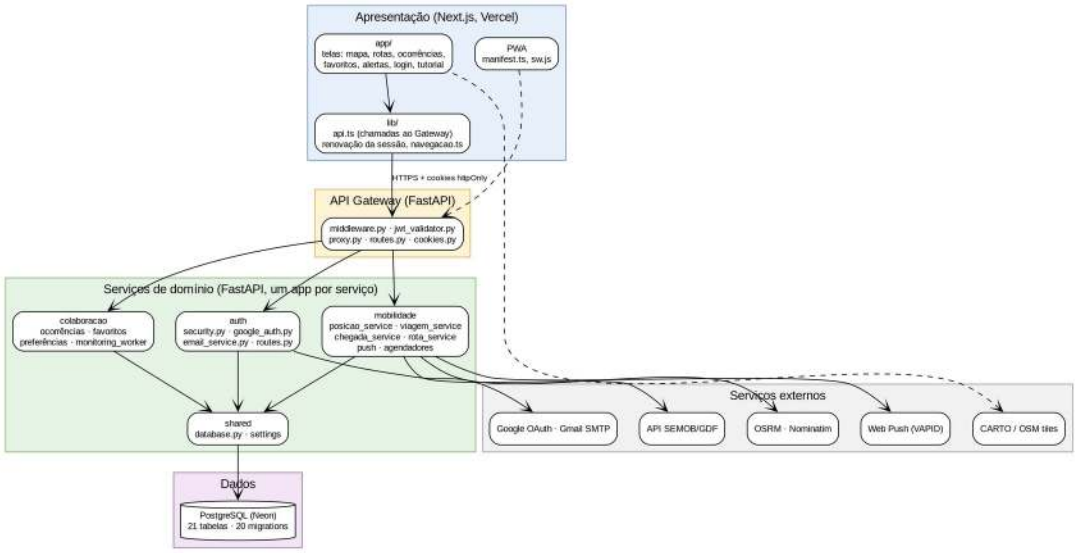


Figura 2 — Diagrama de pacotes da arquitetura lógica do Movecity

Descrição das camadas:

- **Camada de Apresentação (<<interface usuario>>):** Responsável pela interface com o usuário. Implementada em Next.js (React 18, App Router). Contém as páginas, componentes reutilizáveis e o módulo de mapa (Leaflet JS com base CARTO Voyager ou OpenStreetMap). Não contém lógica de negócio — comunica-se exclusivamente com o API Gateway.
- **API Gateway (<<servico gateway>>):** Ponto único de entrada para todas as requisições do frontend. Responsável por rotear chamadas aos microserviços corretos, validar o JWT do cookie localmente (sem round-trip ao serviço de autenticação), gravar e limpar os cookies de sessão e encaminhar a identidade do usuário aos serviços de domínio pelo cabeçalho X-User-Id, descartando o que o cliente enviar com esse nome (ADR-17). A **renovação do token expirado é feita pelo frontend** (ADR-21): o Gateway apenas faz o proxy de POST /api/auth/refresh e regrava os cookies.
- **Serviço de Autenticação (<<servico autenticacao>>):** Gerencia o ciclo de vida de usuários e sessões. Emite e valida credenciais; suporta login por e-mail/senha e OAuth 2.0 (Google, com vínculo de conta existente), cadastro com aceite dos termos, renovação e encerramento de sessão, primeiro acesso (tutorial) e recuperação de senha por e-mail.
- **Serviço de Mobilidade (<<servico mobilidade>>):** Núcleo funcional da plataforma. Lê as posições de GPS do feed da SEMOB, mantém o catálogo de linhas, paradas e trajetos, calcula rotas de origem a destino (com baldeação) apoiado no OSRM, calcula o tempo de chegada, gerencia a viagem do passageiro (embarcar, trocar o destino, encerrar), o alerta de proximidade da parada, o alerta de cancelamento, o planejamento por horário, a posição colaborativa e as inscrições de Web Push.
- **Serviço de Colaboração (<<servico colaboracao>>):** Gerencia o fluxo de reportes colaborativos (ocorrências), a confirmação por outros passageiros (validação cruzada), as rotas favoritas e as preferências de notificação. O monitoring_worker (aviso de rotas preferidas) existe como componente, mas não está agendado em produção (seção 5).
- **Camada de Dados (<<modelo>>):** PostgreSQL (Neon) para dados persistentes — usuários, sessões, catálogo de linhas, rotas, paradas, viagens, ocorrências, favoritos e preferências — e a API externa da SEMOB/GDF para as posições de GPS, lidas ao vivo e nunca persistidas. Cada serviço de domínio acessa o banco por si, com a mesma DATABASE_URL; o Gateway não acessa o banco.

Representação do Funcionamento da Arquitetura

O diagrama de sequência da UML descreve a colaboração temporal entre os objetos do sistema para um determinado cenário. Ele é fundamental para que os desenvolvedores compreendam como as camadas da arquitetura interagem em tempo de execução, especialmente o papel do API Gateway na validação de sessão e no roteamento de requisições.

O cenário escolhido é o **rastreamento de posição de ônibus em tempo real (UC16)** com token JWT válido, pois demonstra o fluxo completo entre todas as camadas da arquitetura:

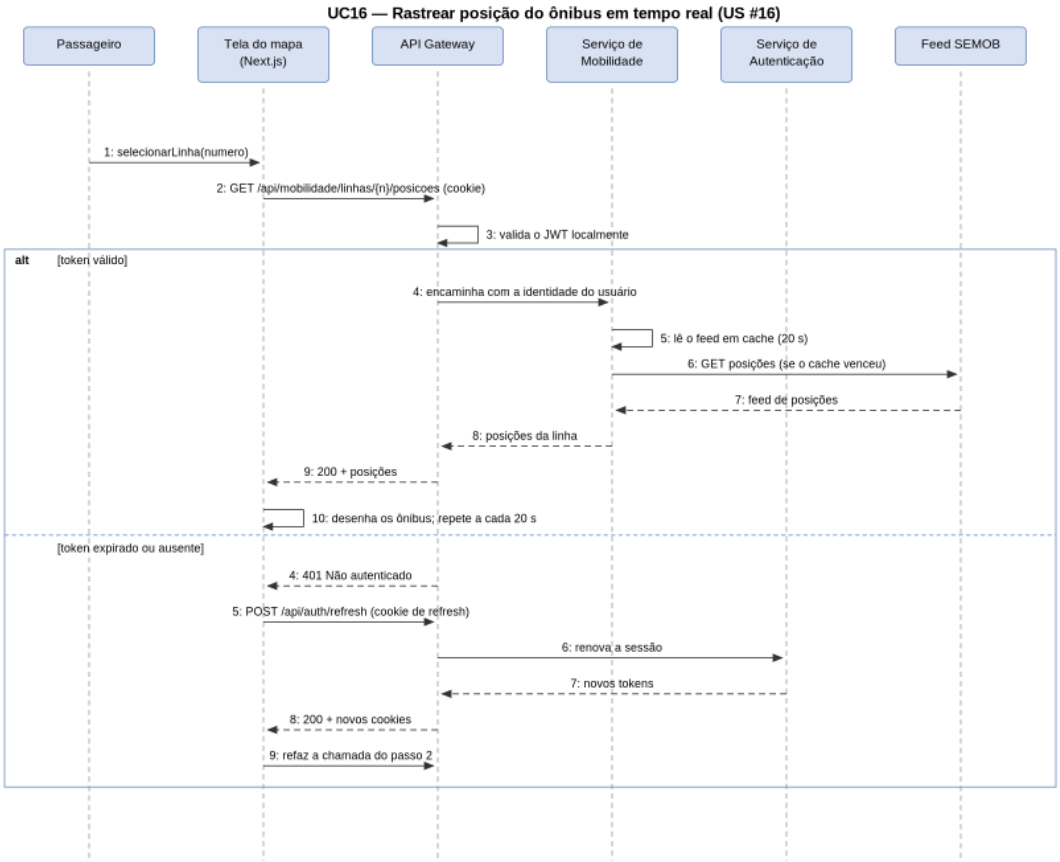


Figura 3 — Diagrama de sequência: UC16 – Rastrear posição do ônibus em tempo real

> *Os diagramas de sequência completos para todos os casos de uso (UC14–UC30) estão disponíveis em docs/diagramas/sequencia/ no formato PlantUML. Eles descrevem o comportamento **planejado**; onde o código divergiu (por exemplo, a renovação do token), vale este documento e docs/gateway-auth-flow.md.*

Visão de Processos

O Movecity separa o tratamento dos dados externos segundo a **frequência com que eles mudam**. Dado estático entra em lote e fica no banco; dado volátil é lido ao vivo e nunca é persistido. Essa distinção define os processos descritos a seguir.

1. Ingestão em lote do catálogo de linhas (fora do ciclo de requisição). Um job agendado no GitHub Actions (.github/workflows/ingestao-semob.yml) baixa os conjuntos /espaciais (1.408 trajetos), /horario e /pontos (7.140 abrigos de parada) da SEMOB — cerca de 37 MB — e popula linha, rota, rota_celula, parada e rota_parada. O job roda mensalmente (todo dia 1º, às 4h UTC) e sob demanda via workflow_dispatch, com limite de 20 minutos; ao final, confere que a tabela linha tem mais de 500 linhas. Os números (1.408 trajetos, 923 linhas, 7.140 abrigos)

estão registrados nos comentários de semob_source.py e em docs/pesquisa-integracao-onibus-df.md.

A ingestão faz a **junção espacial** entre o traçado de cada linha e os abrigos de parada, associando o abrigo à linha quando está a até 40 metros do eixo da via (RAIO_PARADA_METROS). Para evitar a comparação de todos os traçados contra todos os abrigos, é construído um índice de grade em memória (células de 0,0005 grau, cerca de 55 m); a busca examina apenas as 9 células vizinhas de cada ponto.

> *O processamento roda no CI, e não no deploy nem em uma thread do FastAPI, por três motivos: são minutos de execução que não devem repetir-se a cada deploy; dado de linha é estático e não justifica polling; e manter isso fora do contêiner evita que o serviço precise de memória para um pico que ocorre uma vez por mês.*

2. Leitura ao vivo das posições da frota (dentro do ciclo de requisição). O endpoint /posicao da SEMOB devolve a frota inteira do DF (~3.500 veículos, segundo o comentário de posicao_service.py) em uma única resposta, sem aceitar filtro por linha. O posicao_service baixa o feed completo, mantém-no em **cache de processo por 20 segundos** (CACHE_SEGUNDOS) e filtra a linha pedida em memória. Como o cache é do feed inteiro, um download atende simultaneamente todos os usuários e todas as linhas — dentro de cada instância do serviço.

Posições com mais de 15 minutos são descartadas como desatualizadas (IDADE_MAXIMA_MINUTOS) — um veículo em garagem mantém a última posição registrada por dias. **Nenhuma posição é gravada no banco:** o dado perde validade antes de qualquer consulta posterior justificar o custo de escrita. O frontend renova as posições, as chegadas e a viagem por polling (setInterval).

3. Cache sob demanda do detalhe de linha. Quando o usuário busca uma linha ainda não cacheada (ou cujo registro está velho), o LinhaService consulta o LinhaProvider configurado e grava o resultado na tabela linha, com atualizado_em definindo quando o registro deve ser renovado. O provider é o do Google Maps quando GOOGLE_MAPS_API_KEY está configurada e o mock caso contrário (mobilidade/routes.py); em 29/10/2026 a chave **não está** nos secrets de movecity-mobilidade; em produção o provider é o simulado, que serve só de rede de segurança, e o catálogo de linhas vem da tabela linha, populada pela ingestão da SEMOB. Buscas seguintes da mesma linha são servidas do banco, sem tocar a fonte externa.

4. Alerta de proximidade da parada de destino (serviço de mobilidade). No startup, mobilidade/main.py inicia um BackgroundScheduler (APScheduler) **dentro do processo** do serviço. O job monitoramento-proximidade percorre as viagens ativas no banco, localiza o ônibus da viagem na posição ao vivo, calcula o ETA pelo trajeto até a parada de destino, confere as preferências do usuário (notificacoes_ativas, alerta_chegada, antecedência) e, ao cruzar o limiar, grava o alerta em alertas_proximidade — com índice único por (viagem, parada), de modo que o mesmo aviso não se repete — e chama o notificador. O notificador é o NotificadorWebPush quando as três variáveis VAPID existem e são coerentes, e o NotificadorNulo caso contrário (ADR-16); o aviso dentro do app independe do push. O intervalo padrão é o de WORKER_INTERVAL_MINUTES (5 min), ajustável por WORKER_PROXIMIDADE_INTERVAL_SECONDS (mínimo de 20 s). Com 5 minutos, o aviso pode chegar com atraso; a issue #200 prevê medir isso em aparelho e decidir o intervalo — **essa medição não foi feita**.

5. Rastreamento de atraso (#27) — implementado sobre dados simulados. O mesmo agendador executa o job monitoramento-atrasos, que usa o ServicoRastreamento com o FornecedorGTFSMock e o NotificadorNulo. Não há fonte real de atraso nem canal de entrega: o alerta de atraso fica no servidor e não chega ao usuário.

5b. Alerta de cancelamento (#28). O mesmo agendador executa o job monitoramento-cancelamento (ADR-27), com o ServicoCancelamento: acha a partida prevista sem ônibus no feed, grava o alerta, busca a linha alternativa e chama o notificador de Web Push, sempre depois do commit.

5c. Posição colaborativa (#199). POST /viagens/{id}/posicao valida a leitura e a guarda na memória do processo por até 2 minutos; a leitura do mapa e do ETA (PosicaoService) soma esses veículos ao feed da SEMOB, que prevalece (ADR-25).

6. Monitoramento de rotas preferidas (serviço de colaboração) — componente não agendado. O monitoring_worker expõe apenas executar_ciclo() e recebe suas dependências (FavoritosProvider, ETAService, PushSender, Deduplicador) por injeção, o que o mantém testável sem relógio, sem rede e sem banco. Porém

colaboracao/main.py **não tem agendador**: o ciclo só roda quando alguém chama POST /colaboracao/notificacoes/processar, rota de demonstração que usa FavoritosMockProvider, ETAMockProvider e um PushService em memória. As rotas favoritas reais (tabela rotas_favoritas) não alimentam esse ciclo, e nenhum push chega ao usuário por ele.

7. Envio de e-mail em segundo plano. POST /auth/esqueci-senha grava o token e agenda o envio com BackgroundTasks do FastAPI; a resposta não espera o servidor SMTP (ADR-20).

Processo de atendimento a requisições. Cada serviço executa um servidor FastAPI sobre uvicorn. As rotas de proxy do Gateway são assíncronas (httpx.AsyncClient, timeout de 30 s, 504 em timeout e 502 se o serviço não conecta); as demais rotas dos serviços são em sua maioria síncronas e rodam no pool de *threads* do FastAPI. A comunicação entre os serviços é sempre HTTP, via Gateway; o código não contém chamadas diretas entre serviços de domínio nem fila de mensagens.

Visão da Implementação

O código-fonte do Movecity está organizado nos seguintes módulos principais:

Frontend (/src/frontend/):

- app/ — Páginas, layouts e componentes de tela (Next.js App Router). Cada rota concentra seus próprios componentes: app/mapa/ reúne AppShell, MapaInterativo, PlanejadorViagem e os componentes da viagem; as demais rotas são login, cadastro, recuperar-senha, redefinir-senha, tutorial, ocorrencias, favoritos, alertas, configuracoes e status.
- app/globals.css — Fundação visual: *design tokens* da identidade, tipografia e estilos de base compartilhados por todas as telas.
- app/manifest.ts e public/sw.js — Manifesto do PWA e *service worker* (casca do app e recebimento de Web Push).
- lib/ — api.ts (cliente HTTP do Gateway, incluindo fetchAutenticado, que renova a sessão em 401), push.ts (inscrição de Web Push), rotaPendente.ts (rota repassada entre telas sem passar pela URL) e navegacao.ts.
- public/ — Assets estáticos (logotipo, ícones e sw.js).
- __tests__/ — Testes de componente (Jest + Testing Library), colocados ao lado do código que testam.

Backend (/src/backend/): cada serviço é um pacote autocontido e segue a mesma anatomia interna — main.py (aplicação FastAPI), routes.py (endpoints), schemas.py (contratos Pydantic), models/ (entidades SQLAlchemy), providers/ (integrações externas por trás de um Protocol, onde existem) e tests/.

- gateway/ — API Gateway: middleware.py, jwt_validator.py, dependencies.py, proxy.py, cookies.py, config.py, routes.py.
- auth/ — Autenticação: security.py (hash bcrypt e JWT), google_auth.py (OAuth 2.0), email_service.py (e-mail de recuperação de senha por SMTP) e os modelos Usuario, Sessao e TokenResetSenha.
- mobilidade/ — Mobilidade: linha_service.py, rota_service.py, posicao_service.py, eta_service.py, chegada_service.py, geocode_service.py, parada_service.py, busca_parada_service.py, viagem_service.py e viagem_routes.py, web_push.py e push_routes.py, semob_source.py, ingestao_semob.py, workers/monitoramento.py, services/ (rastreamento, proximidade, notificações) e os providers (linha_mock, linha_google_maps, osrm_router, polyline_decoder, gtfs_mock, veiculos_ao_vivo, viagens_ativas_banco, preferencias_banco, entre outros).
- colaboracao/ — Colaboração: routes.py (ocorrências, confirmações, favoritos e preferências de notificação), monitoring_worker.py, eta_service.py, push_service.py, deduplicador.py e os providers eta_mock e favoritos_mock.
- shared/ — Configuração (config.py), sessão de banco (database.py) e filtro de log (logs.py) compartilhados entre os serviços.
- models/base.py — Base declarativa do SQLAlchemy, comum a todos os modelos.

- alembic/ — Migrations versionadas; um único encadeamento (20 revisões em homolog) para todo o banco, mesmo com os serviços separados.

Nota de estrutura: *não existe pacote core/, e o pacote models/ da raiz contém apenas a Base declarativa — os modelos de cada domínio vivem dentro do próprio serviço. A v1.0 deste documento antecipava uma estrutura que a implementação não seguiu.*

Visão de Implantação

O Movecity é implantado em infraestrutura de nuvem, com cada componente em seu ambiente adequado:

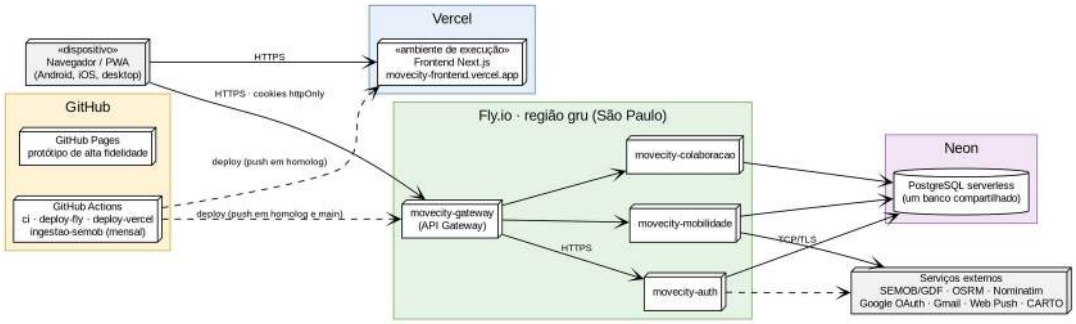


Figura 4 — Diagrama de implantação do Movecity

Nós físicos/lógicos:

Nó	O que executa	Observações
Vercel	Frontend Next.js	Publicado em movecity-frontend.vercel.app. O deploy dispara no push para homolog, em modo --prod: nesta fase, homolog **é** o ambiente público, usado para homologação e demonstrações. main não tem deploy próprio de frontend (ADR-19).
Fly.io — movecity-gateway	API Gateway (FastAPI)	Região primária gru (São Paulo), uma única região. Único serviço que o navegador acessa diretamente (via frontend). Não acessa o banco.
Fly.io — movecity-auth	Serviço de autenticação	Acessa o banco. Recebe GOOGLE_CLIENT_ID, SMTP_USER, SMTP_PASSWORD, SMTP_FROM e FRONTEND_URL como secrets no deploy (os de SMTP só são aplicados se os quatro existirem).
Fly.io — movecity-mobilidade	Serviço de mobilidade	Acessa o banco e as APIs externas (SEMOB, OSRM, Nominatim). Recebe VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY e VAPID_SUBJECT no deploy se existirem; RAIO_EMBARQUE_M é secret do app (ADR-23). Executa o agendador do alerta de proximidade.
Fly.io — movecity-colaboracao	Serviço de colaboração	Acessa o banco.

Nó	O que executa	Observações
Neon (PaaS gerenciado)	PostgreSQL serverless	Um único projeto/banco compartilhado pelos serviços e por homologação e produção (ADR-19). Sem PostGIS. Política de backup e *failover* dependem do plano do Neon e não foram verificadas neste levantamento.
GitHub Pages	Protótipo de alta fidelidade (`prototipo-alta-fid/web`)	Publicado pelo workflow `deploy-pages.yml` em push na `main`. Não faz parte do produto em execução.
SEMOB/GDF, OSRM, Nominatim, CARTO, Google, Gmail, serviços de Web Push	Serviços externos	Não gerenciados pelo Movecity. Cada um tem uma degradação prevista quando indisponível (seção 10.3).

Tabela 3 – Nós de implantação

Todos os apps do Fly têm `internal_port = 8000`, `force_https = true`, `auto_stop_machines = true`, `auto_start_machines = true` e `min_machines_running = 0`. Não há seção `[checks]` nos `fly.toml`: a verificação de saúde é o *smoke test* do pipeline.

Pipeline de implantação (GitHub Actions, `deploy-fly.yml` e `deploy-vercel.yml`):

```
push em homolog ou main (src/backend/**)
      |
      ▼
  [migrate]  alembic upgrade head  ← roda uma vez, antes de tudo
      |
      ├──► [deploy-auth]        ──► secrets (Google, SMTP) ──► deploy ──► smoke test GET /health
      ├──► [deploy-mobilidade]  ──► secrets (VAPID)        ──► deploy ──► smoke test GET /health
(em paralelo)
      └──► [deploy-colaboracao] ──►                        deploy ──► smoke test GET /health

  [deploy-gateway] ──► deploy ──► smoke test GET /health   (independente: não usa banco)

push em homolog (src/frontend/**)
      |
      ▼
  [deploy-homolog]  vercel pull --environment=production → vercel build --prod  → vercel deploy --
prebuilt --prod
```

Cada `deploy-*` tem seu próprio grupo de concorrência, com `cancel-in-progress: false`: dois merges seguidos enfileiram, em vez de um cancelar o deploy do outro pela metade. O **smoke test** consulta `GET /health` do serviço recém-publicado por até 10 tentativas com 5 segundos de intervalo, e falha o job se nenhuma retornar 200 — um contêiner que sobe e morre no boot reprova o deploy em vez de passar despercebido. Os passos de secrets são "aplicar só se existirem": sem os secrets do SMTP ou do VAPID, o deploy conclui e o recurso fica desligado.

Escala a zero. Sem tráfego, o serviço desliga, o que mantém o custo do projeto-piloto próximo de zero, ao preço de uma latência de partida a frio na primeira requisição após um período ocioso — aceitável em avaliação acadêmica, mas incompatível com a meta de 3 segundos da seção 2 se o produto for a público. Essa latência **não foi medida**. Uma consequência adicional: o agendador do alerta de proximidade roda dentro do processo da Mobilidade; com a máquina desligada por ociosidade, ele pode não executar. A mitigação (manter uma máquina viva por serviço) é uma mudança de configuração, não de arquitetura.

Outros fluxos automatizados:

Workflow	Gatilho	Função
ci.yml	Todo PR para homolog ou main e disparo manual (não roda em push)	Quatro jobs: testes de backend (pytest, com migrations, contra um serviço PostgreSQL 16 — a imagem usada é a postgis/postgis, mas nenhuma extensão é usada); verificação de cada serviço em um ambiente limpo, só com o requirements.txt dele, importando a aplicação (ADR-08); testes de frontend (Jest, Node 20); e validação de Conventional Commits no título do PR e nos commits. Não há lint de frontend nem medição de cobertura.
deploy-fly.yml	Push em homolog ou main que altere src/backend/**, e manual	Migrations, deploy dos quatro apps e *smoke test*.
deploy-vercel.yml	Push em homolog que altere src/frontend/**, e manual	Build e deploy de produção do frontend na Vercel.
deploy-pages.yml	Push em main que altere prototipo-alta-fid/web/**, e manual	Publica o protótipo no GitHub Pages.
ingestao-semob.yml	Mensal (dia 1º, 4h UTC) e manual	Baixa os dados da SEMOB e popula as tabelas do catálogo, conferindo ao final que vieram centenas de linhas.

Visão de Dados

O Movecity persiste dados estruturados no PostgreSQL. A seguir, a descrição das entidades principais e seus relacionamentos.

Minimundo: Um Usuario pode ter várias Sessao ativas, um conjunto de PreferenciaNotificacao e múltiplas RotaFavorita. Uma Linha guarda o próprio trajeto e a lista de paradas; a Rota (linha × sentido) liga-se às Parada físicas por RotaParada. Um Usuario autenticado pode criar Ocorrencia vinculada ao **número** de uma linha, e outros usuários podem confirmá-la (ConfirmacaoOcorrencia), incrementando seu contador — regra da validação cruzada (3 confirmações de usuários diferentes do autor). Um Usuario pode ter uma Viagem ativa por vez, com AlertaProximidade e InscricaoPush próprios.

Tabelas implementadas (migrations de homolog):

Tabela	Conteúdo	Origem
usuarios	Conta do usuário: nome, e-mail, hash de senha, provider, google_id, avatar_url, aceite da LGPD, first_access, tentativas falhas e bloqueio, vínculo pendente com Google	US #10–13, #30, #92
sessoes	Sessões emitidas (access e refresh token), para revogação e renovação	US #10, #31, #212
tokens_reset_senha	Tokens de uso único (guardados como hash SHA-256, validade de 60 min) para recuperação de senha	US #13
linha	Catálogo de linhas: número, nome, sentido, paradas, trajeto e horarios_previstos	US #15, #17
rota / rota_celula	Trajeto por linha × sentido, horários por dia da semana, tempo de percurso e o	US #20, #173

Tabela	Conteúdo	Origem
	índice de grade que acelera a busca por proximidade	
parada / rota_parada	Parada física única (código estável) e a posição dela em cada rota: ordem, índice no trajeto, distância acumulada e distância ao traçado	US #173
viagem	Viagem do passageiro, sem coluna de posição; uma ativa por usuário (índice único parcial); compartilha_posicao_em e consentimento_versao guardam só o aceite do compartilhamento	US #171, #199
alertas_cancelamento	Partida prevista sem ônibus: linha, horário, status e linha alternativa; índice único por linha e horário	US #28
alertas_proximidade	Aviso de proximidade disparado por viagem e parada (índice único)	US #172
inscricao_push	Inscrição de Web Push do aparelho: endpoint, chaves públicas do navegador, data da última falha; sem coordenadas, nome ou *user agent*	US #195
ocorrencia	Reporte de ocorrência: número da linha, tipo, descrição, local, lat/lng opcionais, status, contador de confirmações e expira_em	US #23, #24
confirmacoes_ocorrencia	Confirmação por outro usuário; UNIQUE (ocorrencia_id, usuario_id)	US #26
rotas_favoritas	Rota salva: linha, rótulo e coordenadas de origem e destino; limite de 20 por usuário	US #25
preferencias_notificacao	Antecedência do aviso e chaves de ativação por tipo de alerta, uma linha por usuário	US #29
alertas / linhas_acompanhadas	Alerta de atraso e linhas acompanhadas, usados pelo rastreamento sobre dados simulados	US #27
test_user, test_linha, test_log, test_reporte	Tabelas de teste criadas na migration inicial; sem função no produto	—

Tabela 4 – Tabelas persistidas

Entidades não persistidas. PosicaoOnibus **não é persistida por decisão de projeto**: a posição é lida ao vivo e descartada (seção 5, processo 2). A Parada tem tabela própria desde a US #173; linha.paradas segue como JSON só para a exibição da linha.

Integridade referencial entre serviços. Dentro de um serviço, as chaves estrangeiras existem (por exemplo sessoes.usuario_id e tokens_reset_senha.usuario_id para usuarios, confirmacoes_ocorrencia.ocorrencia_id para ocorrencia, rota_parada.parada_id para parada). Entre serviços, o ORM **não declara** a chave para usuarios (ocorrencia, rotas_favoritas, preferencias_notificacao, viagem); nos casos de rotas_favoritas e preferencias_notificacao a migration cria a chave no banco (ADR-22). ocorrencia.linha_numero e viagem.parada_destino_codigo não têm chave estrangeira por desenho.

Representação de geometria. Trajetos e paradas são armazenados em colunas JSON — list[[lat, lng]] para o traçado e list[{nome, lat, lng}] para as paradas —, e não em tipos geoespaciais. O consumidor desses dados é o Leaflet no navegador, que recebe coordenadas em JSON de qualquer forma; guardá-las no formato final evita conversão a cada leitura. A única operação genuinamente espacial do sistema — associar abrigos de parada ao traçado de uma linha — ocorre uma vez por mês, na ingestão, e é resolvida em Python com índice de grade (seção 10, ADR-02). Coordenadas de ocorrências e de favoritos são pares Float (lat/lng), sem tipo geoespacial.

Dados de localização que o banco guarda. A posição de GPS do usuário **não** é gravada (a viagem não tem coluna de posição, ADR-15); a posição colaborativa (ADR-25) vive só em memória, por até 2 minutos. O banco guarda, porém, coordenadas que o usuário escolhe informar: o lat/lng opcional de uma ocorrência e a origem e o destino de uma rota favorita.

Modelo Entidade-Relacionamento (MER):

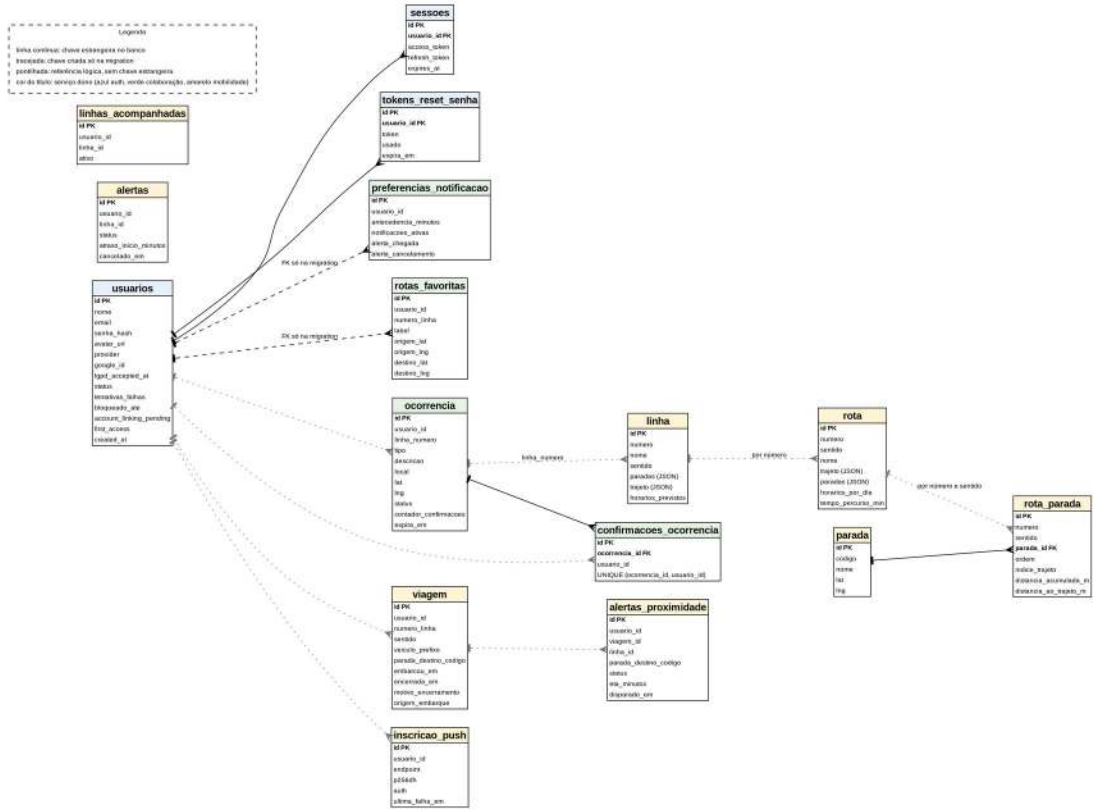


Figura 5 — Modelo entidade-relacionamento (MER) das tabelas implementadas em homolog

> *As coordenadas seguem o sistema de referência WGS84 (o mesmo do GeoJSON da SEMOB/GDF e do Leaflet), porém guardadas como pares numéricos em colunas JSON, sem tipo geoespacial nem índice espacial no banco.*

Volume e Desempenho

Esta seção separa o que é **premissa de dimensionamento** (nunca medida) do que é **fato verificado** no código ou nos dados.

Premissas do projeto-piloto (Ceilândia e Taguatinga) — não medidas, sem teste de carga:

Métrica	Valor assumido
Usuários cadastrados (fase piloto)	5.000
Usuários simultâneos (média)	500

Métrica	Valor assumido
Pico de usuários simultâneos (horários de pico)	1.500
Requisições por segundo (pico)	150 req/s
Disponibilidade mínima	99% nos horários de pico (5h–9h e 17h–20h)
Tempo de carregamento inicial	≤ 3 segundos em conexão 4G

Fatos verificados:

Item	Valor	Fonte
Linhas no catálogo (ingestão SEMOB)	923 linhas / 1.408 trajetos	semob_source.py
Abrigos de parada no catálogo	7.140	semob_source.py
Volume baixado por ingestão	~37 MB	ingestao-semob.yml
Atualização do feed de GPS	Leitura ao vivo, cache de 20 s por instância; posição com mais de 15 min é descartada	posicao_service.py
Retenção de posições GPS	Não persistidas	ADR-04
Retenção de ocorrências	O registro **não é apagado**; o expira_em (15 a 120 min, conforme o tipo) só o tira da listagem	colaboracao/models/ocorrencia.py
Validade da sessão	Access token de 1 h; refresh token de 7 dias	gateway/cookies.py, auth/security.py
Validade do link de recuperação de senha	60 minutos, uso único	auth/models/token_reset_senha.py
Duração máxima de uma viagem	4 h (depois expira)	viagem_service.py
Backup	Não verificado (depende do plano do Neon)	—

> ***Ressalva sobre as estimativas de volume.*** *Os números de crescimento de dados foram projetados na v1.0 supondo um histórico de posições de GPS em banco. Com a ADR-04 (posição lida ao vivo e não persistida), o crescimento real do banco é governado pelo catálogo de linhas — estático — e pelas tabelas de usuário, o que o torna substancialmente menor que o projetado. As metas de desempenho e disponibilidade seguem como metas,* ***não como resultados****: nem o tempo de carregamento, nem a disponibilidade, nem a capacidade foram medidos, e a ADR-12 registra o risco da partida a frio.*

Evolução Arquitetural (junho a outubro de 2026)

Esta seção registra as decisões arquiteturais tomadas **durante a implementação**, entre a versão 1.0 deste documento (12/jun/26) e 31/out/26. Cada registro segue o formato ADR (*Architecture Decision Record*): a decisão, o que a motivou e a consequência assumida. Onde a decisão contraria a v1.0, isso está dito explicitamente. As ADR-01 a ADR-15 foram registradas na v2.0 e conferidas contra o código na v3.0; as ADR-16 a ADR-24 são da v3.0 e as ADR-25 a ADR-27, da v3.2.

Registro de decisões (ADR-01 a ADR-15)

ID	Decisão	Motivação	Consequência assumida
ADR-01	O Gateway alcança os serviços pelas **URLs públicas HTTPS** (movecity-*.fly.dev), e não pela rede privada .internal do Fly.io.	A rede privada do Fly é IPv6-only. Para usá-la, os serviços precisariam de bind dual-stack, o que causou instabilidade com o proxy público do próprio Fly.	Os serviços ficam acessíveis pelas URLs públicas; o Gateway é o único ponto que o frontend chama.
ADR-02	**PostGIS não foi adotado.** Geometrias ficam em colunas	O produto tem uma só operação espacial de verdade	**Contraria a v1.0**, que previa PostGIS em várias seções. Se

ID	Decisão	Motivação	Consequência assumida
	JSON; a única junção espacial é resolvida em Python com índice de grade (células de ~55 m, raio de busca de 40 m).	— associar abrigos de parada ao traçado da linha — e ela roda uma vez por mês, na ingestão. Adotar PostGIS acrescentaria uma extensão, um tipo de coluna e um vocabulário de consulta para atender a um caso único e offline.	surgir busca por raio em tempo de requisição (ex.: "paradas perto de mim" no servidor), a decisão deve ser reavaliada — hoje esse cálculo acontece no navegador.
ADR-03	Os dados da SEMOB são **copiados para o banco** por ingestão em lote, em vez de consultados a cada busca.	Os endpoints não aceitam filtro: devolvem sempre o payload inteiro (~37 MB somados). Consultar por usuário seria inviável. Além disso, são endpoints públicos sem documentação nem SLA publicados.	Se a SEMOB sair do ar, a busca continua funcionando com o último snapshot. Em troca, o catálogo pode ficar até um mês desatualizado — aceitável para dado que muda raramente.
ADR-04	Posições de GPS são **lidas ao vivo com cache de 20 s e nunca persistidas**.	O feed traz a frota inteira do DF em uma resposta só; um download serve todos os usuários simultaneamente. Posição perde validade em segundos.	**Contraria a v1.0**, que previa persistir posições a cada 30 s com retenção de 24 h. Não há histórico de posições — se análises retrospectivas entrarem no escopo, será preciso um processo de arquivamento à parte.
ADR-05	O traçado exibido no mapa passa pelo **OSRM**, e não pela linha reta entre paradas.	A geometria bruta ligava paradas em linha reta, atravessando quadras. O OSRM ajusta o traçado à malha viária real do OpenStreetMap.	Mais uma dependência externa no caminho do usuário. O trajeto é cacheado junto com a linha, então a chamada não se repete a cada visualização.
ADR-06	Integrações externas ficam atrás de um **contrato `typing.Protocol`**, com implementação mock e implementação real.	Permite desenvolver e testar sem rede e sem cota de API, e trocar a fonte em um ponto só.	Foi o que permitiu migrar do mock para os dados reais do SEMOB sem reescrever `LinhaService`, rota HTTP nem teste. Custo: uma camada de indireção a mais em cada integração.
ADR-07	Cookies de sessão usam **`SameSite=None` com `Secure`**, e não `Lax`/`Strict`.	Frontend (Vercel) e Gateway (Fly) estão em domínios diferentes — é cross-site de fato. Com `Lax`/`Strict`, o navegador não envia o cookie em `fetch()` cross-site, e toda chamada autenticada caía em 401 logo após um login bem-sucedido.	**Corrige a v1.0**, cuja tabela de cookies indicava `Lax`/`Strict`. `SameSite=None` amplia a exposição a CSRF; a mitigação atual é o escopo restrito de `path` por cookie e a obrigatoriedade de HTTPS. Unificar frontend e Gateway sob um mesmo domínio permitiria voltar a `Lax`.
ADR-08	Cada serviço tem seu **próprio `requirements.txt`**, além do combinado na raiz.	Cada imagem Docker instala só o que o serviço usa, reduzindo o tamanho da imagem e o tempo de build.	Criou uma armadilha real: o CI instala o combinado da raiz, e o contêiner instala o do serviço. Dependência ausente no arquivo do serviço **passa no CI e derruba o contêiner no boot**. Aconteceu duas

ID	Decisão	Motivação	Consequência assumida
			vezes (PR #102, no mobilidade; PR #127/#128, no colaboração). O smoke test em /health no pipeline existe justamente para transformar esse erro em deploy reprovado.
ADR-09	O CORSMiddleware é o **middleware mais externo** da pilha do Gateway.	O Starlette executa primeiro o último middleware adicionado. Com o CORS abaixo do middleware de autenticação, o preflight OPTIONS — que nunca carrega cookie — era barrado com 401 antes de o CORS responder.	Todo 401 legítimo passa a chegar ao frontend como 401, e não como "bloqueado por CORS". A ordem virou regra verificada por teste (test_cors_ordem_middleware.py).
ADR-10	As **migrations rodam no pipeline**, em um job único que antecede os deploys dos serviços.	Com quatro serviços e um banco só, deixar cada contêiner migrar no boot causaria corrida entre eles.	Um encadeamento único do Alembic para todo o banco, mesmo com os serviços separados — o que exige atenção ao down_revision quando duas USs criam migration em paralelo (aconteceu na US #29).
ADR-11	A base do mapa é o **CARTO Voyager**, com recuo automático para o tile padrão do OpenStreetMap.	O tile padrão do OSM é denso e saturado, e disputa atenção com o que importa na tela. A base do CARTO tem contraste menor e deixa legíveis linha, parada e veículo.	A CARTO passou a exigir chave. A chave é gratuita (5 milhões de tiles/mês), mas virou uma variável de ambiente a mais. Sem ela, o mapa recua para o OSM e continua funcionando — nenhuma tela quebra por falta de credencial.
ADR-12	As máquinas do Fly operam com **escala a zero** (min_machines_running = 0).	Mantém o custo do projeto-piloto próximo de zero.	Partida a frio na primeira requisição após ociosidade. Conflita com a meta de 3 segundos da seção 2 em um cenário real de produção; a correção é configuração, não arquitetura.
ADR-13	O proxy do Gateway repassa a **query string** e **não repassa o Accept-Encoding** do cliente.	Descartar a query string quebrava toda rota com parâmetro. Repassar o Accept-Encoding fazia o serviço de destino devolver corpo comprimido que o proxy entregava sem descomprimir.	O proxy deixou de ser transparente por acidente e passou a ser transparente por contrato, com teste cobrindo os dois comportamentos.
ADR-14	A **parada física tem tabela própria** (parada), criada por fusão de abrigos a até 10 m na ingestão, com código derivado do centróide e desempate de colisão.	O /pontos do SEMOB não tem identificador e repete o mesmo abrigo em registros vizinhos (~900 pares a até 15 m). O código derivado da coordenada clicada mudava conforme o registro, e o ETA	A tabela é reconstruída a cada ingestão, então o id muda; **quem consome deve usar o codigo**. O espaço de 100 mil códigos colide (113 de 4.318 paradas), por isso há sondagem determinística.

ID	Decisão	Motivação	Consequência assumida
		não conseguia relacionar parada, veículo e sentido.	Dois abrigos de lados opostos da rua (> 10 m) continuam separados. Fusão a 10 m junta sentidos exclusivos opostos em 1 caso de 307 medidos.
ADR-15	A **viagem do passageiro é persistida** (`viagem`), mas **sem nenhuma coluna de posição**: guarda usuário, linha, sentido, prefixo do veículo, parada de destino e os horários de embarque e encerramento. O embarque é confirmado no servidor por proximidade (veículo da linha a até 100 m, posição de até 5 min) e a posição do usuário enviada para isso é descartada. O progresso (próxima parada, paradas restantes) é calculado na leitura, a partir da posição ao vivo do veículo.	O produto precisa saber que alguém está a bordo (para esconder a espera e disparar o alerta de proximidade da #172), mas a LGPD e a ADR-04 vedam trilha de posição. Inferir o embarque por GPS contínuo exigiria rastrear a pessoa o tempo todo.	Uma viagem ativa por usuário, garantida por **índice único parcial** no banco (não só na aplicação). Viagem esquecida expira em 4 h. A tabela guarda `parada_destino_codigo` **sem chave estrangeira**, porque `parada` é reconstruída a cada ingestão (ADR-14); destino que sumiu simplesmente deixa a viagem fora do alerta. O raio padrão de 100 m passou a ser configurável por variável de ambiente e a ter uma segunda regra de embarque (ADR-23); calibrá-lo em campo continua pendente: com o feed atualizando a cada 20 s, um ônibus em movimento já andou ~220 m entre duas leituras.

Tabela 5 – Registro de decisões arquiteturais

Decisões da Sprint 3 e da Sprint 4 (ADR-16 a ADR-27)

ID	Decisão	Motivo	Consequência
ADR-16	**Web Push com chaves VAPID** para o alerta de proximidade, enviado pelo serviço de Mobilidade (`pywebpush`). A inscrição do aparelho fica na tabela `inscricao_push`, sem coordenadas, nome nem *user agent*; o dono vem do `X-User-Id`, nunca do corpo; só são aceitos endpoints `https` de FCM, Mozilla, Apple e Windows (proteção contra SSRF). Sem as três variáveis VAPID, o notificador é nulo e o controle aparece como indisponível.	O alerta só aparecia dentro do app; quem está com a tela bloqueada não era avisado (#195).	O push cobre o alerta de proximidade; o aviso dentro do app independe dele. As chaves valem para homologação e produção (ADR-19), e trocá-las invalida as inscrições. O ciclo do worker (padrão de 5 min) pode atrasar o aviso.
ADR-17	**O Gateway descarta o `X-User-Id` enviado pelo cliente** (sem diferenciar maiúsculas, mesmo em rota	O cabeçalho do cliente também chegava ao serviço (#188).	A identidade que o serviço lê é sempre a do Gateway; o contrato é coberto por testes do Gateway.

ID	Decisão	Motivo	Consequência
	pública) e repassa só a identidade extraída do JWT.		
ADR-19	**Homologação e produção não são separadas.** O mesmo pipeline publica os quatro apps do Fly em push para `homolog` ou `main`, há um único banco Neon, e o frontend publica só a partir de `homolog`, em modo `--prod`.	Custo: a separação foi retirada do escopo pelo grupo (#42, fechada em 14/10).	`homolog` é o ambiente visível ao usuário: um merge muda o que as pessoas veem, as migrations rodam no banco compartilhado e as variáveis de ambiente valem para os dois lados.
ADR-20	**E-mail de recuperação de senha por SMTP** (Gmail, porta 587, STARTTLS), com `smtplib`, em segundo plano (`BackgroundTasks`). O token vale 60 min, é de uso único e guardado como hash SHA-256; a resposta é a mesma para e-mail cadastrado e não cadastrado; limite de 5 solicitações por e-mail em 15 min. **Não há confirmação de conta por e-mail.**	O envio era só um esboço que registrava o link no log, e a recuperação (#13, #133) nunca entregava o e-mail.	Depende de uma conta Gmail; falha de SMTP é registrada e não derruba o endpoint. O limite de solicitações vive na memória do processo.
ADR-21	**Rota `POST /auth/refresh` e renovação pelo frontend.** O Auth valida o refresh token, confere a sessão, **rotaciona** o par de tokens e responde; o Gateway regrava os cookies. No frontend, `fetchAutenticado` renova **uma única vez** ao receber 401 e repete a requisição; renovações concorrentes compartilham a mesma promessa. Se falha, leva a `/login?sessao=expirada`.	O Gateway fazia proxy do refresh, mas o Auth não tinha a rota (#212), e o frontend tratava o 401 como "sem dados" (#221).	A renovação é reativa, não agendada, e o Gateway só valida o access token e faz proxy do refresh.
ADR-22	**A chave estrangeira para `usuarios` sai do ORM de Colaboração** e fica no banco, pela migration. Um teste de regressão carrega só os modelos de Colaboração.	O modelo `Usuario` pertence ao Auth e não existe no contêiner de Colaboração; sem ele, `GET /colaboracao/preferencias/{id}` respondia 500 (#213).	O banco garante a integridade; o `autogenerate` do Alembic pode propor remover essas chaves e não deve ser usado para "corrigir" a diferença.
ADR-23	**Regra de embarque configurável:** `RAIO_EMBARQUE_M` (padrão 100 m) e a regra do "ônibus chegando" (usuário a até 400 m de uma parada da linha e ônibus a até 2 min dela). A posição do usuário só valida o embarque e é descartada (ADR-15); a do veículo tem no máximo 5	Quem estava na parada com o ônibus chegando era recusado (#190).	**Em produção o raio está em 100 km, por decisão do grupo, só para demonstrar o "Peguei este ônibus" na defesa; a regra de uso real é 100 m.** Os limiares não foram validados em campo.

ID	Decisão	Motivo	Consequência
	min.		
ADR-24	**A posição do usuário não vai para a URL:** a rota pendente passa por sessionStorage (chave movecity:rota-pendente), apagada na primeira leitura.	As coordenadas do botão "Ir" iam para o histórico, links copiados e logs (#231), contrariando a LGPD.	A rota não sobrevive à aba nem se compartilha por link. O filtro shared/logs.py já omitia a query string do log.
ADR-25	**Fallback colaborativo da posição do ônibus (#199), só em memória.** Com consentimento explícito (desligado por padrão, separado do aceite dos Termos, revogável), o passageiro a bordo envia a posição do ônibus (POST /viagens/{id}/posicao, 202). Quando a linha não tem ônibus equivalente no feed da SEMOB, essa leitura entra como veículo colaborativo no ETA, no mapa e na parada, com o selo "colaborativo". A SEMOB sempre vence, inclusive no empate. As leituras vivem no processo, por no máximo 2 minutos, atrás de uma interface de armazenamento (ArmazenamentoPosicoesColaborativas); o banco só ganha viagem.compartilha_posicao_em e consentimento_versao. Leitura fora do traçado (150 m), com salto de velocidade impossível ou com menos de 10 s de intervalo é recusada.	O "ônibus fantasma" também acontece quando o ônibus roda e não aparece no GPS oficial. Gravar a posição do passageiro contrariaria a LGPD e a ADR-04.	Nenhuma coordenada vai para o banco ou para o log. **Limite:** a memória é por processo; o serviço de mobilidade tem duas máquinas no Fly com escala a zero, então a leitura enviada a uma máquina pode não ser vista por quem consulta na outra. Trocar por armazenamento compartilhado é trocar a implementação da interface. O consentimento só pode ser dado no embarque.
ADR-26	**Planejamento por horário (#115) como camada sobre a rota da #20, sem GTFS nem RAPTOR.** GET /mobilidade/planejar recebe tipo (saida ou chegada), horario e data, pega as opções de RotaService.calcular (uma ou duas pernas) e aplica a tabela de saídas do dia da semana (rota.horarios_por_dia) e o tempo de percurso. O horário de passagem em cada	A issue previa feed GTFS e um motor de roteamento temporal, o que traria um serviço novo para hospedar. As opções da #20 são poucas, então uma camada de horário por cima resolve o caso do projeto-piloto.	O horário é **estimado**, não de uma tabela de passagem por parada (a resposta traz estimado=true e a tela avisa). Feriados não são considerados: a SEMOB não informa. Não há busca em grafo espaço-temporal; combinações com mais de uma baldeação ficam fora.

ID	Decisão	Motivo	Consequência
	ponto é a saída do terminal mais o tempo de percurso proporcional à posição do ponto no traçado (o mesmo princípio de passagem_prevista). "Sair às X" pega o primeiro ônibus possível em cada perna; "chegar até X" calcula de trás para frente. A espera na baldeação e a caminhada entram na conta, e espera acima de 3 h não é viagem. Sem opção, a resposta traz motivo e mensagem.		
ADR-27	**Alerta de cancelamento (#28) por ausência do ônibus.** O agendador da Mobilidade ganha o job monitoramento-cancelamento, que varre as linhas acompanhadas, localiza as partidas previstas que passaram entre 15 e 45 minutos atrás (linha.horarios_previstos, fuso de Brasília) e confere se a linha tem veículo no feed. Sem veículo, e com outras linhas reportando posição no mesmo ciclo, grava o alerta em alertas_cancelamento (índice único por linha e horário), busca uma linha alternativa que cubra ao menos 70% das paradas e notifica por Web Push quem ativou esse tipo de alerta.	A SEMOB não informa cancelamento operacional. Só a ausência do ônibus permite detectar o "ônibus fantasma".	Não distingue cancelamento de atraso longo ou de falha do feed; por isso o feed inteiro vazio não gera alerta. O job vive no processo da Mobilidade e herda a limitação de escala a zero (seção 7).

Tabela 5.1 – Decisões da Sprint 3 e da Sprint 4

Degradação prevista por dependência externa

Uma consequência de somar serviços externos ao caminho do usuário é que a indisponibilidade de cada um precisa ter resposta definida. O quadro abaixo registra o comportamento atual:

Dependência	Se ficar indisponível	Comportamento do sistema
SEMOB /espaciais, /horario, /pontos	Ingestão mensal falha	Busca de linha continua servindo o último snapshot do banco.
SEMOB /posicao	Sem posições ao vivo	O mapa exibe o trajeto e as paradas; o rastreamento informa que não há veículo com posição recente — mesma tela do caso, normal, em que a linha não tem veículo em viagem.
OSRM	Sem ajuste à malha viária	O trajeto já cacheado continua sendo

Dependência	Se ficar indisponível	Comportamento do sistema
		exibido; linhas novas ficam sem traçado ajustado.
Nominatim	Sem busca de endereço no planejador	A busca de lugares falha; a busca de linha e de parada pelo catálogo não depende dele.
CARTO	Sem base visual do mapa	Recuo automático para o tile padrão do OpenStreetMap.
Google OAuth	Sem login social	Login por e-mail e senha permanece disponível.
Gmail SMTP	Sem e-mail de recuperação	O endpoint responde normalmente; o e-mail não chega e o erro só aparece no log (ADR-20).
Serviços de Web Push	Sem aviso com a tela bloqueada	O aviso de proximidade continua dentro do app (ADR-16).
Chaves VAPID ausentes	Push desligado	Mesmo comportamento do item anterior; o controle em Configurações aparece como indisponível.

Tabela 6 – Degradação prevista por dependência externa

Limites conhecidos da arquitetura atual

Registro do que o código mostra e ainda não foi tratado. Não são decisões, são lacunas assumidas:

1. **Logout não invalida o access token.** O Gateway valida o JWT só pela assinatura e pela expiração, sem consultar sessoes. O logout apaga a sessão e, com isso, impede o refresh, mas um access token já emitido continua aceito até expirar (1 h). O logout localiza a sessão pelo cookie access_token (auth/routes.py); se o navegador já descartou esse cookie por vencimento, o código não apaga a sessão.
2. **Estado em memória por processo:** cache de posições, limite de solicitações de recuperação de senha e o ciclo de demonstração da Colaboração não são compartilhados entre instâncias.
3. **Segredos de produção** ficam como secrets de cada app no Fly.io e não são verificáveis a partir do repositório.
4. **Sem teste automatizado de ponta a ponta** entre frontend, Gateway e serviços; o que cobre isso é a execução manual em produção.
5. **Homologação e produção compartilhadas** (ADR-19).

Divergências entre a v1.0 e a implementação

Previsões da v1.0 que não se confirmaram, e o registro delas é parte do valor deste documento:

1. **PostGIS e persistência de posições (ADR-02 e ADR-04).** A v1.0 dimensionou o banco para armazenar posições de GPS em tipos geoespaciais, com retenção de 24 horas. A implementação não persiste posição alguma, e a única operação espacial saiu do banco. As estimativas de volume da seção 9 devem ser lidas com essa ressalva: o crescimento projetado de ~2 GB/mês pressupunha um histórico de posições que não existe.
2. **Estrutura de pacotes do backend (seção 6).** A v1.0 previa models/ e core/ centralizados. A implementação organizou modelos e configuração por serviço, com apenas a Base declarativa e a configuração compartilhada na raiz — coerente com o isolamento pretendido pelo estilo de microserviços.
3. **Ingestão como thread do FastAPI (seção 5).** A v1.0 previa um worker assíncrono dentro do backend. A ingestão foi para o CI. Os agendadores que restaram no backend rodam dentro do processo da Mobilidade (alerta de proximidade e rastreamento de atraso simulado); o monitoring_worker da Colaboração não tem agendador.
4. **Renovação de sessão pelo Gateway (seção 4.3).** A v1.0 e a v2.0 atribuíam ao Gateway a renovação transparente do token. Na implementação, o Gateway só faz proxy do refresh e regrava os cookies; quem

detecta o 401 e renova é o frontend (ADR-21).

5. **Nenhum serviço acessa o banco.** O texto original do bloco "Gateway como Proxy" dizia que nenhum serviço backend se comunica com o banco e que tudo passa pelo Gateway. Na implementação, Auth, Mobilidade e Colaboração acessam o PostgreSQL diretamente; o Gateway é quem não acessa.
6. **Validação cruzada com 2 reportes (seção 2).** A v1.0 previa o mínimo de 2 reportes independentes. O código exige 3 confirmações de usuários diferentes do autor para confirmar ocorrências sujeitas à validação, e exibe acidente e segurança de imediato.
7. **Retenção de ocorrências de 30 dias (seção 9).** O código não apaga ocorrências; o campo `expira_em` (15 a 120 min) só as retira da listagem.
8. **Disponibilidade em múltiplas regiões do Fly (seção 2).** Cada app roda em uma só região (`gru`).

Referências

CAROLI, Paulo. **Lean Inception: como alinhar pessoas e construir o produto certo**. São Paulo: Caroli.org, 2018.

GDF — GOVERNO DO DISTRITO FEDERAL. **Secretaria de Mobilidade Urbana (SEMOB)**. API de rastreamento de frota do STPC/DF. Disponível em: portal de dados abertos do GDF. Acesso em: jun. 2026.

IPEDF — INSTITUTO DE PESQUISA E ESTATÍSTICA DO DISTRITO FEDERAL. **Pesquisa Distrital por Amostra de Domicílios — PDAD 2022**. Brasília: IPEDF, 2022.

ISO/IEC 25010:2011. **Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models**. Geneva: ISO, 2011. Adotada como ABNT NBR ISO/IEC 25010.

KRUCHTEN, Philippe. The 4+1 view model of architecture. **IEEE Software**, v. 12, n. 6, p. 42–50, nov. 1994.

CGDF — CONTROLADORIA GERAL DO DISTRITO FEDERAL. **Painel de Ouvidoria do Distrito Federal**. Brasília: CGDF, 2026.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.